

# **Comparative Analysis of Various Sorting Algorithms**



Siluni Dulvanya Pasqual

Department of Electronic & Telecommunication Engineering

Faculty of Engineering

## **Abstract**

Sorting algorithms are fundamental in computer science, serving as the backbone of numerous applications that require efficient data organization and retrieval. This research aims to explore a comprehensive range of sorting algorithms, encompassing comparison-based approaches such as Bubble Sort, Selection Sort, and Merge Sort, alongside non-comparison-based techniques like Counting Sort and Radix Sort. Each algorithm is evaluated based on implementation, computational complexity (time complexity), memory usage (space complexity), and practical efficiency. By comparing these algorithms across various performance parameters and input sizes, the study identifies their suitability for different contexts. Additionally, graphical and tabular representations are provided to illustrate the experimental findings. The research highlights the critical role of sorting algorithms in optimizing data handling processes and aims to guide the selection of appropriate sorting techniques for diverse applications.

## Table of Contents

|        |                                                                                   |    |
|--------|-----------------------------------------------------------------------------------|----|
| 1      | BACKGROUND OF THE PROJECT.....                                                    | 1  |
| 1.1    | Introduction.....                                                                 | 1  |
| 1.2    | Classification of Sorting Algorithms .....                                        | 1  |
| 1.3    | Aim of the project .....                                                          | 3  |
| 1.4    | Objectives of the project .....                                                   | 3  |
| 1.5    | Scope of the project .....                                                        | 4  |
| 2      | LITERATURE REVIEW .....                                                           | 4  |
| 2.1    | Performance Analysis of Various Sorting Algorithms .....                          | 4  |
| 2.2    | Performance Analysis of Merge Sort Algorithms .....                               | 4  |
| 2.3    | Performance Measurement of Popular Sorting Algorithms using Java and Python ..... | 5  |
| 3      | METHODOLOGY .....                                                                 | 5  |
| 3.1    | Algorithm Selection.....                                                          | 6  |
| 3.2    | Experimenting the selected algorithms .....                                       | 6  |
| 3.2.1  | Quick Sort .....                                                                  | 6  |
| 3.2.2  | Randomized Quick Sort.....                                                        | 10 |
| 3.2.3  | Merge Sort .....                                                                  | 11 |
| 3.2.4  | Heap Sort .....                                                                   | 14 |
| 3.2.5  | Bubble Sort .....                                                                 | 17 |
| 3.2.6  | Insertion Sort.....                                                               | 20 |
| 3.2.7  | Selection Sort.....                                                               | 22 |
| 3.2.8  | Counting Sort.....                                                                | 24 |
| 3.2.9  | Radix Sort .....                                                                  | 27 |
| 3.2.10 | Bucket Sort.....                                                                  | 29 |
| 3.3    | Analysis of various Sorting Algorithms.....                                       | 31 |
| 3.3.1  | Big O notation.....                                                               | 31 |
| 3.3.2  | Time Complexity and Space Complexity of Various Sorting Algorithms.....           | 32 |
| 4      | RESULTS & DISCUSSION.....                                                         | 33 |
| 4.1    | Experimental Analysis .....                                                       | 33 |
| 4.1.1  | Case 1: Running Time of $O(n \log n)$ sorting algorithms .....                    | 33 |
| 4.1.2  | Case 2: Running Time of $O(n^2)$ sorting algorithms.....                          | 34 |

|       |                                                                      |    |
|-------|----------------------------------------------------------------------|----|
| 4.1.3 | Case 3: Running Time of non-comparison-based sorting algorithms..... | 34 |
| 4.2   | Discussion .....                                                     | 35 |
| 4.2.1 | Comparison of $O(n \log n)$ sorting algorithms .....                 | 35 |
| 4.2.2 | Comparison of $O(n^2)$ sorting algorithms.....                       | 36 |
| 4.2.3 | Comparison of Non-Comparison-Based Sorting Algorithms .....          | 37 |
| 5     | CONCLUSION.....                                                      | 38 |
|       | REFERENCES .....                                                     | 59 |

## Table of Figures & Tables

|                 |                                                                          |    |
|-----------------|--------------------------------------------------------------------------|----|
| Figure 3.2.3.1  | Mergesort example for a single array .....                               | 11 |
| Figure 3.2.3.2  | Python code for the mergesort example considering a single array .....   | 12 |
| Figure 3.2.4.1  | Heapsort example for a single array .....                                | 15 |
| Figure 3.2.4.2  | Python code for the heapsort example considering a single array .....    | 16 |
| Figure 3.2.5.1  | Bubble sort array example for a single array.....                        | 18 |
| Figure 3.2.5.2  | Python code for the Bubble sort example considering a single array ..... | 18 |
| Figure 3.2.6.1  | Insertion sort example for a single array .....                          | 20 |
| Figure 3.2.6.2  | Python code for Insertion sort considering a single array .....          | 21 |
| Figure 3.2.7.1  | Selection sort example for a single array .....                          | 23 |
| Figure 3.2.7.2  | Python code for Insertion sort considering a single array .....          | 23 |
| Figure 3.2.8.1  | Counting Sort example for a single array.....                            | 25 |
| Figure 3.2.8.2  | Python code for Counting sort considering a single array .....           | 25 |
| Figure 3.2.9.1  | Radix Sort example for a single array .....                              | 27 |
| Figure 3.2.9.2  | Python code for Radix sort considering a single array.....               | 28 |
| Figure 3.2.10.1 | Bucket sort example for a single array .....                             | 30 |
| Figure 3.2.10.2 | Python code for Bucket sort considering a single array .....             | 30 |
| Figure 4.1.1.1  | Graph of comparison of $O(n \log n)$ sorting algorithms.....             | 33 |
| Figure 4.1.2.1  | Graph of comparison of $O(n^2)$ sorting algorithms.....                  | 34 |
| Figure 4.1.3.1  | Graph of comparison of non-comparison-based sorting algorithms .....     | 35 |
| Table 1:        | Analysis of various sorting algorithms .....                             | 32 |
| Table 2:        | Comparison of $O(n \log n)$ sorting algorithms.....                      | 33 |
| Table 3:        | Comparison of $O(n^2)$ sorting algorithms.....                           | 34 |
| Table 4:        | Comparison of non-comparison-based sorting algorithms .....              | 34 |

# 1 BACKGROUND OF THE PROJECT

## 1.1 Introduction

Sorting, often referred to as the ordering of data, involves arranging the elements of a list in a specific sequence, typically in ascending or descending order. The process transforms the input into a reordered output, also known as a permutation. Sorting algorithms are among the most fundamental tasks in computer science, playing a critical role in optimizing various applications. For instance, searching in a sorted list is significantly faster and more efficient compared to an unsorted list, as it allows advanced search techniques like binary search. By organizing data in a sorted manner, we can achieve highly optimized search operations, saving both time and computational resources.

A practical example of sorting can be seen in a dictionary, where words are stored in alphabetical order. This systematic arrangement makes it effortless to locate any word, demonstrating how sorting simplifies real-world tasks. Whether in databases, e-commerce platforms, or scientific computations, sorting algorithms form the backbone of efficient data processing and retrieval.

## 1.2 Classification of Sorting Algorithms

Sorting algorithms can be classified based on various parameters, each highlighting different aspects of their behavior, efficiency, and implementation. These classifications help in understanding the strengths and weaknesses of each algorithm and guide the selection of the most suitable one for a given problem. Below are the key parameters used to classify sorting algorithms:

### 1. By number of comparisons:

Sorting algorithms can be categorized based on the number of comparisons they perform. Comparison-based sorting algorithms, such as Merge Sort and Quick Sort, rely on comparing elements to determine their order. For such algorithms, the best-case time complexity is  $O(n \log n)$ , while the worst-case can be  $O(n^2)$ . It has been proven that comparison-based algorithms require at least  $O(n \log n)$  comparisons for most inputs.

## 2. By memory usage:

Sorting algorithms are classified based on their memory usage patterns. **In-place** sorting algorithms, like Heap Sort and Insertion Sort, use a constant amount of additional memory ( $O(1)$ ) beyond the input data. In contrast, **out-of-place** sorting algorithms, such as Merge Sort, require additional memory proportional to the input size ( $O(n)$ ). While in-place algorithms are memory-efficient, they may be slower compared to out-of-place algorithms.

## 3. By recursion:

Sorting algorithms can be either recursive or non-recursive. Recursive algorithms, like Quick Sort and Merge Sort, solve the problem by breaking it into smaller subproblems and solving them recursively. Non-recursive algorithms, such as Insertion Sort and Selection Sort, use iterative approaches. Some algorithms, like Merge Sort, combine both recursive and iterative techniques.

## 4. By stability:

A sorting algorithm is considered **stable** if it maintains the relative order of elements with equal keys. For example, if two elements  $A[0]$  and  $A[1]$  have the same value and  $A[0]$  appears before  $A[1]$  in the unsorted list, a stable algorithm will ensure  $A[0]$  remains before  $A[1]$  in the sorted list. Examples of stable algorithms include Merge Sort and Bubble Sort, while Quick Sort and Heap Sort are generally unstable.

## 5. By adaptability:

Adaptive sorting algorithms adjust their behavior based on the initial order of the input. For example, if the input is already partially or fully sorted, adaptive algorithms like Insertion Sort and Bubble Sort can achieve better time complexity ( $O(n)$ ). Non-adaptive algorithms, such as Selection Sort, do not benefit from pre-sorted inputs and maintain the same complexity regardless of the input order.

## 6. By Internal vs External Sorting:

- **Internal Sorting:** Algorithms that sort data entirely within the main memory (RAM) are called internal sorting algorithms. These algorithms, such as Quick Sort, Insertion Sort, and Heap Sort, assume fast, random access to all memory locations.

- **External Sorting:** When the data to be sorted is too large to fit into main memory, external sorting algorithms are used. These algorithms, like Merge Sort, utilize external storage (e.g., disks) and are generally slower due to higher access times.

### 1.3 Aim of the project

The aim of this project is to conduct a comprehensive analysis of various sorting algorithms, exploring their implementation, computational efficiency, and real-world applications. By evaluating these algorithms based on time complexity, space complexity, stability, and adaptability, the project seeks to provide valuable insights into their performance across diverse datasets. This study also aims to identify the optimal sorting algorithms for different scenarios, offering a reference for selecting the most efficient technique for specific use cases in data organization and processing.

### 1.4 Objectives of the project

1. **To analyze the performance** of various sorting algorithms in terms of time and space complexity.
2. **To compare the efficiency** of comparison-based and non-comparison-based sorting techniques across different datasets.
3. **To identify the most suitable** sorting algorithms for specific real-world applications based on their strengths and limitations.
4. **To provide a comprehensive evaluation** of sorting algorithms through **experimental results** and visual representations.

## 1.5 Scope of the project

The scope of this project encompasses a comprehensive study of various sorting algorithms, including both comparison-based (e.g., Bubble Sort, Quick Sort) and non-comparison-based (e.g., Counting Sort, Radix Sort) techniques. It involves analyzing their performance in terms of time complexity, space complexity, stability, and adaptability, with experimental validation on real-world datasets. The project highlights the strengths and limitations of these algorithms, explores their applications in areas like database management and search optimization, and provides clear visual representations of the results. Ultimately, it aims to serve as a resource for understanding sorting algorithms and offers guidelines for selecting the most suitable techniques for specific applications.

## 2 LITERATURE REVIEW

The study of sorting algorithms has been extensive, with numerous research efforts dedicated to analyzing their performance, efficiency, and applicability. This literature review highlights three significant IEEE research papers that contribute to this field.

### 2.1 Performance Analysis of Various Sorting Algorithms

This research provides a comprehensive analysis of the practical performance of popular sorting algorithms, **focusing on time complexity**. The study evaluates algorithms such as **Quick Sort, Merge Sort, and Heap Sort** across different data sets and conditions. The findings offer insights into the efficiency of these algorithms in various scenarios, aiding in the selection of appropriate sorting techniques for specific applications.

### 2.2 Performance Analysis of Merge Sort Algorithms

This research focuses on the Merge Sort algorithm, presenting a comparative performance evaluation of five **different merge sorting algorithms**: serial, parallel, bitonic, odd-even, and modified merge sort. The study examines their performance in terms of speed and power consumption, providing valuable insights into the efficiency of each variant in different contexts.

## 2.3 Performance Measurement of Popular Sorting Algorithms using Java and Python

This research presents a thorough asymptotic analysis and performance measurement of popular sorting algorithms **using Python and Java**. The study evaluates the algorithms' performance across various input sizes and data distributions, offering insights into their **efficiency and scalability**. The findings assist in understanding the practical implications of implementing these algorithms in different programming environments.

## 3 METHODOLOGY

### Overview of the Approach and Its Importance

This study adopts a systematic approach to evaluate and analyze various sorting algorithms, ensuring a comprehensive understanding of their performance and applicability. The methodology involves selecting a diverse set of sorting algorithms, implementing them consistently, and testing them across multiple datasets with varying characteristics, such as size and data distribution. Performance metrics, including time complexity, space complexity, stability, and adaptability, are used to assess each algorithm's efficiency.

A systematic approach is critical for maintaining consistency and reliability in the evaluation process. It allows for objective comparisons between algorithms and ensures that findings are reproducible. By carefully designing experiments and analyzing results through graphical and tabular representations, this study aims to provide actionable insights into the strengths, limitations, and optimal use cases for different sorting techniques. This structured evaluation not only highlights the practical applications of sorting algorithms but also guides their selection for specific computational tasks.

### 3.1 Algorithm Selection

- **Comparison based algorithms:** In this type, the elements are sorted by comparing them with one another.
  - Quicksort
  - Merge Sort
  - Bubble Sort
  - Insertion Sort
  - Heap Sort
  - Selection Sort
- **Non-comparison-based algorithms:** In non-comparison algorithms, they are not sorted based on comparing with other elements. Instead, each algorithm uses its own approach.
  - Counting
  - Radix Sort
  - Bucket Sort

### 3.2 Experimenting the selected algorithms

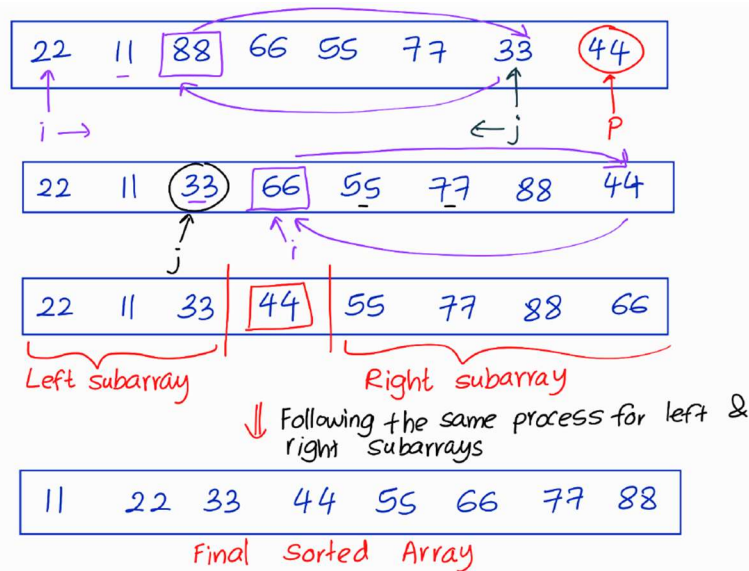
#### 3.2.1 Quick Sort

Quicksort is a divide-and-conquer algorithm that works by selecting a **pivot element** from the array and partitioning the other elements into two sub-arrays - one with elements less than the pivot and one with elements greater. These sub-arrays are then recursively sorted. Quicksort is efficient with an **average case time complexity of  $O(n \log n)$**  but its **worst-case complexity is  $O(n^2)$** , making it less efficient if the pivot selection is poor.

#### Concept:

1. Choose a pivot element (typically the last element in the array).
2. Rearrange the array so that all elements smaller than the pivot are on its left, and all elements greater are on its right. The pivot is then placed in its correct position, dividing the array into two subarrays.
3. Apply the same process recursively to the left and right subarrays until the entire array is sorted.

Ex: - Take a single array => [22, 11, 88, 66, 55, 77, 33, 44]



Quick sort example

Figure: 3.2.1.1 Quick Sort example for a single array

```
def quicksort(arr, left, right):
    if left < right:
        partition_pos = partition(arr, left, right)
        quicksort(arr, left, partition_pos - 1)
        quicksort(arr, partition_pos + 1, right)

def partition(arr, left, right):
    i = left
    j = right - 1
    pivot = arr[right]

    #Checking whether the i and j are crossed
    while i < j:
        while i < right and arr[i] < pivot:
            i += 1
        while j > left and arr[j] >= pivot:
            j -= 1
        if i < j:
            arr[i], arr[j] = arr[j], arr[i]

    if arr[i] > pivot:
        arr[i], arr[right] = arr[right], arr[i]

    return i

arr = [22, 11, 88, 66, 55, 77, 33, 44]
quicksort(arr, 0, len(arr) - 1)
print(arr)
```

Figure 3.2.1.2 Python code for the quicksort example considering a single array

## Implementation for larger arrays

### 1. Generating 3 different random arrays

To ensure fair testing, **three** distinct random arrays were generated for each input size: **1000, 5000, 10000, 20000, 50000, and 100000**. These arrays **remained consistent** across all comparison based sorting algorithms, ensuring that performance differences were solely due to the algorithms themselves rather than variations in input data. This approach provided a reliable basis for comparison and minimized bias in the evaluation of execution times.

### 2. Measuring execution times for different input sizes

The sorting function was executed on each array size, and the time taken for completion was recorded. To ensure reliable results, execution times were averaged across multiple runs, reducing the impact of any anomalies or fluctuations in system performance. This approach provided a more accurate representation of each algorithm's efficiency.

### 3. Storing and analyzing results

The recorded execution times were plotted in a line chart to visually compare performance across different sorting algorithms.

I've included one code for this under the Appendix section with an array size of 1000, and for the other datasets with different input sizes, I adjusted the array size parameter to 5000, 10000, 20000, 50000, and 100000. The implementation includes both standard Quicksort and a variation with randomized pivot selection. In the standard approach, the pivot is typically chosen as the last or first element, while in the randomized version, the pivot is selected randomly to improve performance on already sorted or nearly sorted data. Execution time for each sorting run was measured and recorded, with multiple runs averaged to ensure accuracy. The results are summarized in a table displaying execution times for each input size, and a line chart is included under the Results and Discussion section to visually compare the performance of Quicksort with other sorting algorithms.

## Advantages

- Quicksort has an average-case time complexity of  $O(n \log n)$ , making it one of the fastest sorting algorithms for handling large datasets. It performs significantly better than  $O(n^2)$  algorithms like bubble sort or insertion sort, especially as the number of elements increases.
- Unlike merge sort, which requires additional memory to store temporary arrays, Quicksort sorts elements within the same array. This reduces space complexity and makes it a more memory-efficient sorting technique.
- The algorithm breaks the problem into smaller subproblems by partitioning the array around a pivot. This recursive nature allows Quicksort to efficiently reduce the sorting problem into manageable pieces, leading to faster execution.

## Disadvantages

- If the pivot is poorly chosen, such as always selecting the smallest or largest element, Quicksort's performance can degrade to  $O(n^2)$ . This can occur when sorting already sorted or reverse-sorted arrays, leading to significantly slower execution.
- Quicksort does not preserve the relative order of equal elements unless additional modifications are made. This can be a disadvantage when sorting data that requires maintaining the original sequence of identical values.
- While Quicksort is highly efficient for large datasets, it may not be the best choice for smaller arrays. Simple sorting algorithms like insertion sort often perform better for small input sizes due to their lower overhead and straightforward implementation.

### 3.2.2 Randomized Quick Sort

To minimize the likelihood of encountering the worst-case scenario in Quicksort, randomization can be introduced into the algorithm. This is achieved by selecting a pivot element at random from the given input data instead of always choosing a fixed position (e.g., first or last element). In the randomized version of Quicksort, a randomly selected element from the subarray **A[low..high]** is used as the pivot, ensuring a more balanced partitioning and improving overall performance.

#### Advantages

- By selecting a pivot randomly, the likelihood of encountering the worst-case scenario ( $O(n^2)$  complexity) is significantly reduced. This makes the algorithm more efficient on average, especially for nearly sorted or identical data sets.
- Randomized pivot selection helps ensure more balanced partitions, leading to improved performance in practical applications, particularly when sorting large arrays.

#### Disadvantages

- The process of selecting a random pivot introduces slight additional computational overhead compared to a fixed pivot selection method, which may have a small impact on performance in some cases.
- Although randomization reduces the chances of hitting the worst-case time complexity, it does not completely eliminate it. In rare scenarios, the random pivot selection may still lead to highly unbalanced partitions.

### 3.2.3 Merge Sort

Mergesort is a divide-and-conquer algorithm that works by recursively dividing the array into two halves until each sub-array contains a single element. Then, these sub-arrays are merged back together in a sorted manner. The merging process ensures that the final array is sorted by comparing elements from both halves and placing them in order. Mergesort has a consistent time complexity of  $O(n \log n)$  in all cases, making it efficient for large datasets. However, it requires additional memory for storing temporary sub-arrays, which can be a disadvantage compared to in-place sorting algorithms.

#### Concept:

1. Recursively split the array into two halves until each sub-array contains only a single element or is empty.
2. Merge the sub-arrays by comparing elements from both halves and placing them in order to form a sorted sequence.
3. Continue merging sorted sub-arrays until the entire array is reconstructed and fully sorted.

Ex: - Take a single array => [2, 6, 5, 1, 7, 4, 3]

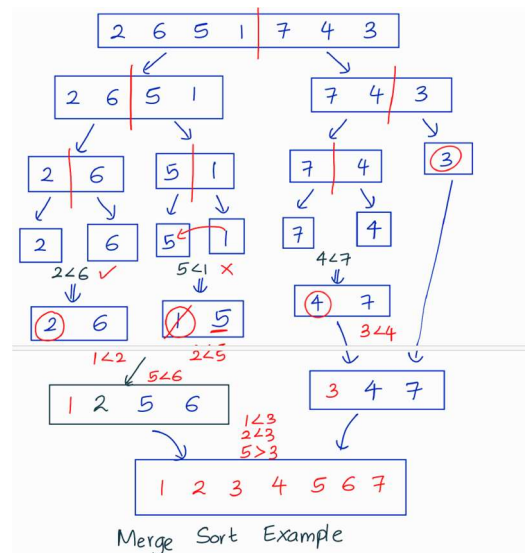


Figure 3.2.3.1 Mergesort example for a single array

```

def merge_sort(arr):
    if len(arr)>1: #If arr=1 it is already sorted
        left_arr = arr[:len(arr)//2]
        right_arr = arr[len(arr)//2:]

        #recursion
        merge_sort(left_arr)
        merge_sort(right_arr)

        #merge
        i=0 #left array index
        j=0 #right array index
        k=0 #merged array index

        while i<len(left_arr) and j<len(right_arr):
            if left_arr[i]<right_arr[j]:
                arr[k] = left_arr[i]
                i+=1
            else:
                arr[k] = right_arr[j]
                j+=1
            k+=1

        while i<len(left_arr):
            arr[k] = left_arr[i]
            i+=1
            k+=1

        while j<len(right_arr):
            arr[k] = right_arr[j]
            j+=1
            k+=1

arr_test = [2, 3, 5, 1, 7, 4, 4, 2, 6, 0]
merge_sort(arr_test)
print(arr_test)

```

*Figure 3.2.3.2 Python code for the mergesort example considering a single array*

### **Implementation for larger arrays**

For implementing Merge Sort on larger arrays, I followed a similar process as for Quicksort, using input sizes of 1000, 5000, 10000, 20000, 50000, and 100000. The same three distinct random arrays which were generated in the quicksort were used for consistency across different sorting algorithms. For each of these array sizes, the Merge Sort algorithm was executed, and the time taken for execution was recorded. The execution times were averaged across multiple runs to ensure reliable and consistent results.

The core of the implementation remained the same for each array size, but the input size was adjusted accordingly for each dataset. As the array size increased, the recursive calls in Merge Sort naturally grew in number, leading to a longer execution time. By maintaining the same random arrays across all sorting algorithms, a fair comparison of performance across the different algorithms was ensured. The changes in the parameters, like the array size, allowed to observe how Merge Sort's time complexity scales with larger inputs.

(The codes and the graphs can be seen under the Appendix Section and the Results and Discussion Section).

### **Advantages**

- Merge Sort consistently performs with a time complexity of  $O(n \log n)$  in both the worst and average cases, which ensures reliable performance even with large datasets.
- Merge Sort is a stable sorting algorithm, meaning that it maintains the relative order of equal elements, which is important when sorting data that contains multiple equal values (e.g., sorting a list of people by age while keeping their names in the same order).

### **Disadvantages**

- Merge Sort requires additional space proportional to the size of the input array ( $O(n)$ ), which can be inefficient in terms of memory usage, especially when working with large arrays.
- Although Merge Sort is efficient for large datasets, it tends to be slower than algorithms like Quicksort or Insertion Sort for smaller arrays due to its additional overhead of recursive calls and space usage.

### 3.2.4 Heap Sort

Heap Sort is a comparison-based sorting algorithm that works by organizing the input data into a binary heap data structure, which is a complete binary tree. The heap can be either a max-heap (where the parent node is greater than or equal to its children) or a min-heap (where the parent node is smaller than or equal to its children). Heap Sort uses a max-heap for sorting in ascending order.

#### Time Complexity:

- Best, Worst, and Average Case:  $O(n \log n)$
- Space Complexity:  $O(1)$  (in-place sorting)

Heap Sort is an efficient, in-place sorting algorithm, **ideal for applications where space complexity is critical**. However, it does not provide stable sorting and can be slower than other  $O(n \log n)$  algorithms like Merge Sort and Quick Sort in practical scenarios.

#### Concept:

1. The first step is to convert the input array into a max-heap, where the largest element is at the root of the tree.
2. Once the max-heap is built, the largest element (root) is swapped with the last element in the heap (the last element in the array).
3. After the swap, the heap property may be violated, so the heap is re-heapified by adjusting the new root and ensuring the heap property is maintained.
4. The heap size is reduced by one (since the largest element is now sorted), and steps 2 and 3 are repeated until all elements are sorted.

Ex: - Take an array  $\Rightarrow [5, 16, 8, 14, 20, 1, 26]$

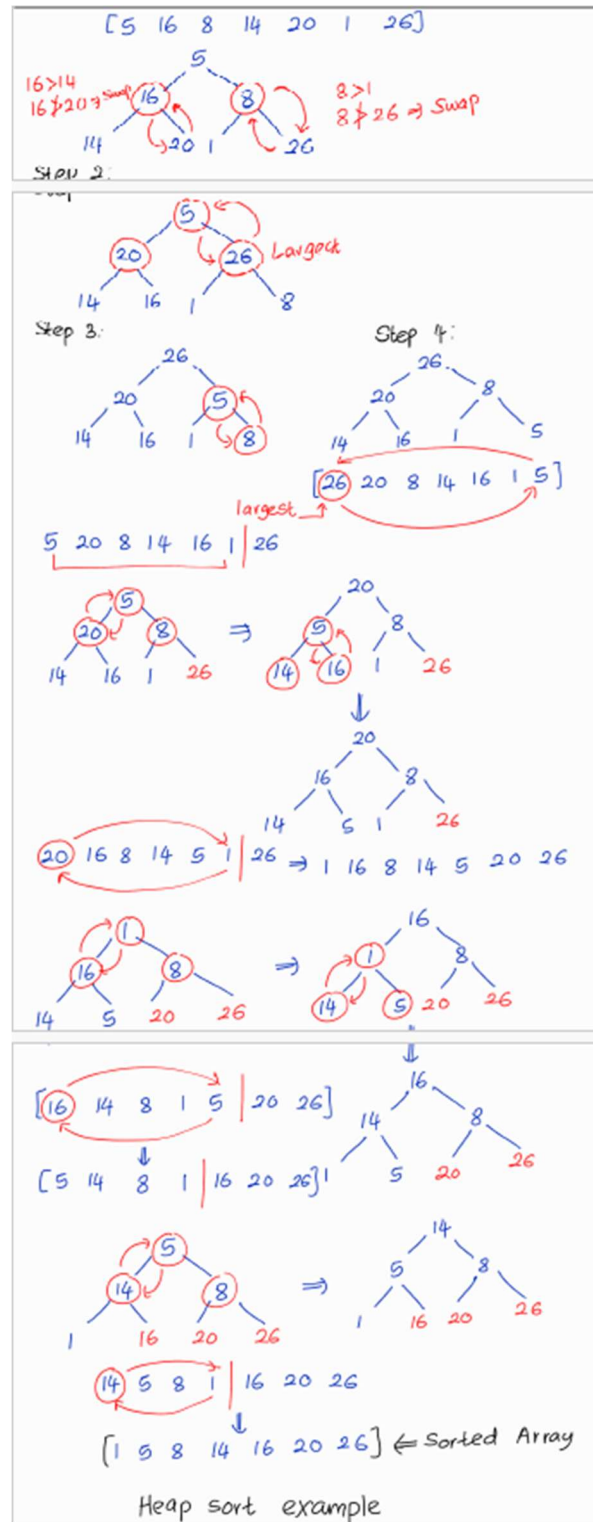


Figure 3.2.4.1 Heapsort example for a single array

```

def swap(lst, i, j):
    lst[i],lst[j] = lst[j], lst[i]

def siftDown(lst,i,upper):
    while(True):
        l,r = i*2+1,i*2+2
        if max(l,r)<upper:
            if lst[i]>=max(lst[l],lst[r]): break
            elif lst[l]>lst[r]:
                swap(lst,i,l)
                i=l
            else:
                swap(lst,i,r)
                i=r

        elif l<upper:
            if lst[l]>lst[i]:
                swap(lst,i,l)
                i=l
            else: break
        elif r<upper:
            if lst[r]>lst[i]:
                swap(lst,i,r)
                i=r
            else: break
        else: break

def heapsort(lst):
    for j in range(len(lst)-2//2,-1,-1):
        siftDown(lst,j,len(lst))

    for end in range(len(lst)-1,0,-1):
        swap(lst,0,end)
        siftDown(lst,0,end)

lst = [5,16,8,14,20,1,26]
heapsort(lst)
print(lst)

```

*Figure 3.2.4.2 Python code for the heapsort example considering a single array*

### Implementation for larger arrays

To test Heap Sort on larger datasets, arrays of sizes **1000, 5000, 10000, 20000, 50000, and 100000** were generated. The same randomly generated arrays were used which were generated for quicksort, across all sorting algorithms to ensure a fair comparison. The implementation of Heap Sort remained the same for all input sizes, with only the size of the array being adjusted.

(The codes and the graphs can be seen under the Appendix Section and the Results and Discussion Section).

## Advantages

- Unlike QuickSort, which has a worst-case complexity of  $O(n^2)$ , Heap Sort always runs in  $O(n \log n)$ , making it more reliable for large datasets.
- Heap Sort does not require additional memory apart from the input array itself (except for a few auxiliary variables), making it more space-efficient than Merge Sort, which requires  $O(n)$  extra space.

## Disadvantages

- Although Heap Sort has a guaranteed  $O(n \log n)$  time complexity, its constant factors are higher due to repeated swapping operations, making it generally slower than QuickSort for most practical applications.
- Heap Sort does not preserve the relative order of duplicate elements, making it an unstable sort. If stability is required, other algorithms like Merge Sort or a modified version of QuickSort are preferred.

### 3.2.5 Bubble Sort

Bubble Sort is a simple comparison-based sorting algorithm that repeatedly steps through the list, compares adjacent elements, and swaps them if they are in the wrong order. This process continues until the entire list is sorted. The algorithm makes multiple passes through the list, with each pass ensuring that the next largest element is placed in its correct position. While Bubble Sort is easy to understand and implement, it is highly inefficient for large datasets due to its  $O(n^2)$  time complexity. As the input size increases, the number of required comparisons and swaps grows significantly, making it one of the slowest sorting algorithms for large data sets.

Ex: - Take an array  $\Rightarrow$  [8, 3, 5, 10, 7]

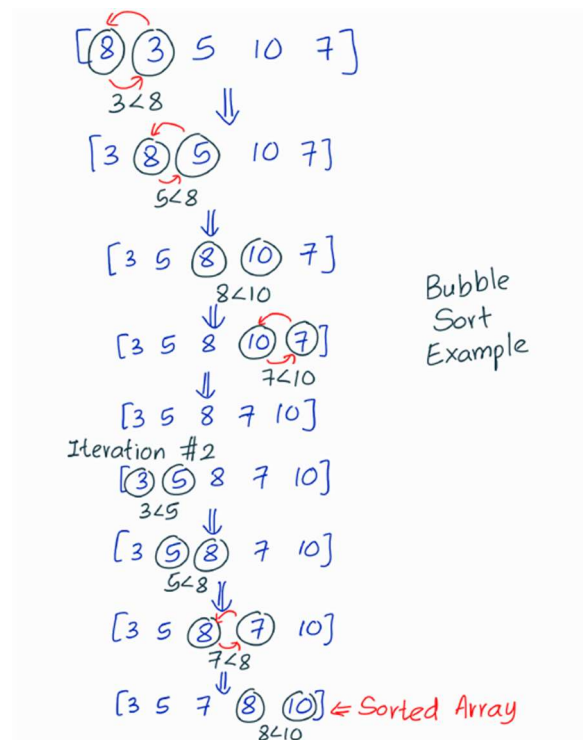


Figure 3.2.5.1 Bubble sort array example for a single array

```
def bubble(list_a):
    indexing_length = len(list_a) - 1
    sorted = False

    while not sorted: #Repeat until sorted = True
        sorted = True

        for i in range(0, indexing_length):
            if list_a[i] > list_a[i+1]:
                sorted = False
                list_a[i], list_a[i+1] = list_a[i+1], list_a[i] #Switch these values
        return list_a

print(bubble([8, 3, 5, 10, 7]))
```

Figure 3.2.5.2 Python code for the Bubble sort example considering a single array

### Implementation for larger arrays

To implement Bubble Sort for larger arrays, **three distinct random arrays** were generated for each input size: **1000, 5000, 10000, 20000, 50000, and 100000**. These same arrays were used across all  **$O(n^2)$  sorting algorithms** (Bubble Sort, Optimized Bubble Sort, Insertion Sort, and Selection Sort) to ensure a fair comparison. The Bubble Sort function was executed on each dataset, and execution times were recorded. To improve reliability, multiple runs were conducted, and the **average execution time** was calculated. Due to its  **$O(n^2)$  time complexity**, Bubble Sort's performance significantly degraded with larger inputs, making it the slowest algorithm among the tested sorting methods.

### Advantages

- Bubble Sort is one of the easiest sorting algorithms to understand and implement, making it ideal for educational purposes and learning basic sorting concepts.
- Bubble Sort is an in-place sorting algorithm, meaning it does not require extra memory beyond the input array, making it space-efficient with  $O(1)$  space complexity.

### Disadvantages

- With a worst-case and average time complexity of  $O(n^2)$ , Bubble Sort performs very poorly for large input sizes, making it impractical for real-world applications.
- Even if the array is nearly sorted, Bubble Sort still performs many unnecessary comparisons and swaps, leading to inefficient execution compared to other sorting algorithms like Quicksort or Merge Sort.

### 3.2.6 Insertion Sort

Insertion Sort is a simple and efficient comparison-based sorting algorithm that works similarly to how we sort playing cards in our hands. The algorithm iterates through the list, picking one element at a time and inserting it into its correct position by comparing it with the already sorted part of the array. This process continues until the entire list is sorted. Insertion Sort is an in-place algorithm, meaning it does not require extra memory beyond the input array. While it performs well on small or nearly sorted datasets, its average and worst-case time complexity is  $O(n^2)$ , making it inefficient for large datasets.

#### Concept:

1. Start with the second element and compare it with the first.
2. If it is smaller, shift the first element to the right and insert the second element in its correct position.
3. Move to the next element and compare it with the sorted part of the array, shifting elements as needed.
4. Repeat this process until the entire array is sorted.

Ex: - Take an array [3, 2, 5, 7, 4]

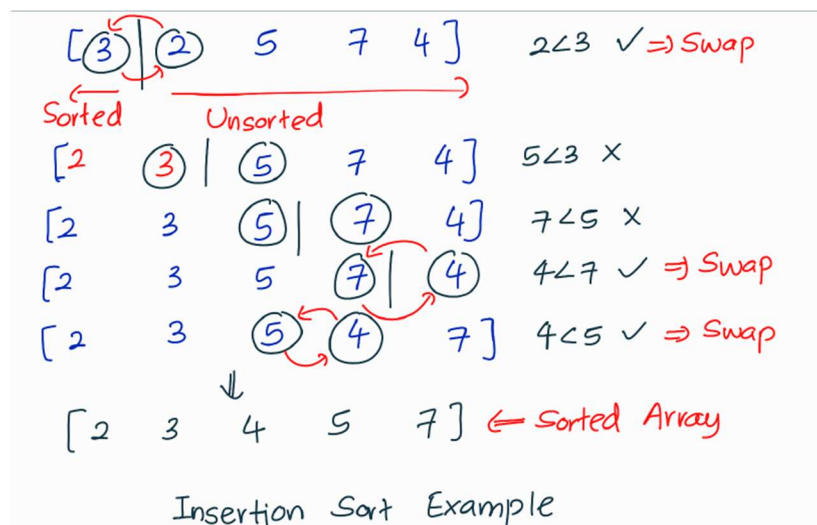


Figure 3.2.6.1 Insertion sort example for a single array

```
def insertion_sort(list_a):
    indexing_length = range(1, len(list_a)) #This takes all the values except the first one.

    for i in indexing_length:
        value_to_sort = list_a[i]

        while list_a[i-1]>value_to_sort and i>0:
            list_a[i],list_a[i-1] = list_a[i-1], list_a[i]
            i=i-1

    return list_a

print(insertion_sort([3, 2, 5, 7, 4]))
```

*Figure 3.2.6.2 Python code for Insertion sort considering a single array*

### Implementation for larger arrays

To implement insertion sort for larger arrays, three distinct random arrays of sizes 1000, 5000, 10000, 20000, 50000, and 100000 were generated. These same arrays were used across all  $O(n^2)$  sorting algorithms to maintain consistency in comparison. The insertion sort function was applied to each array, and the execution time was recorded. The process was repeated multiple times, and the average execution time was taken to ensure accuracy in the results.

### Advantages

- Insertion sort is straightforward and requires minimal code, making it easy to implement for small datasets or educational purposes.
- It performs well on small datasets and is highly efficient when the input array is already or nearly sorted, with a time complexity close to  $O(n)$ .
- Insertion sort does not require additional memory, as it sorts the array within the given space, making it a space-efficient algorithm.
- It maintains the relative order of equal elements, which is useful for applications where order preservation is important.

## Disadvantages

- Insertion sort has an average and worst-case time complexity of  $O(n^2)$ , making it inefficient for large datasets compared to other sorting algorithms like Quicksort or Merge Sort.
- As the input size increases, the number of swaps and comparisons grows significantly, making the algorithm computationally expensive.
- When dealing with large, randomly ordered data, insertion sort becomes impractical due to its quadratic time complexity.

### 3.2.7 Selection Sort

Selection sort is a simple comparison-based sorting algorithm that works by repeatedly finding the smallest (or largest) element from the unsorted portion of the array and swapping it with the first unsorted element. This process is continued until the entire array is sorted. Since selection sort performs a fixed number of comparisons, its time complexity is  $O(n^2)$  in all cases, making it inefficient for large datasets. However, it is an in-place sorting algorithm, requiring only a constant amount of extra memory.

#### Concept:

1. Start from the first element and scan the entire array to find the smallest element.
2. Swap the smallest element found with the first element of the unsorted portion.
3. Move to the next position and repeat the process for the remaining unsorted elements until the entire array is sorted.

Ex: - Take an array  $\Rightarrow [4, 6, 2, 8, 7]$

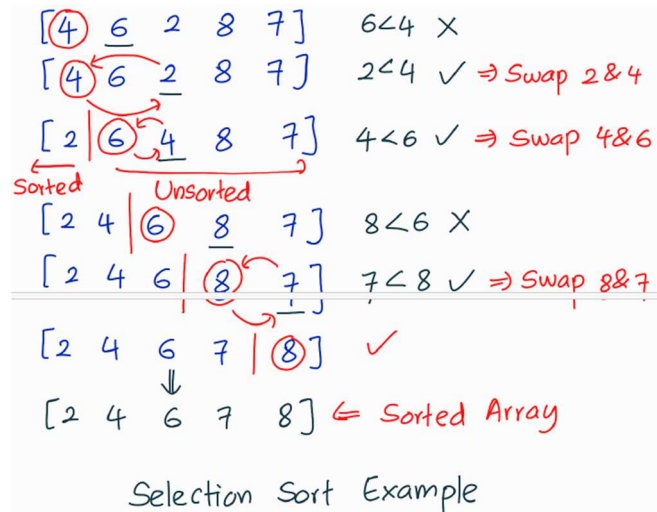


Figure 3.2.7.1 Selection sort example for a single array

```
def selection_sort(list_a):
    indexing_length = range(0, len(list_a)-1)

    for i in indexing_length:
        min_value = i

        for j in range(i+1, len(list_a)):
            if list_a[j] < list_a[min_value]:
                min_value = j

        if min_value != i:
            list_a[min_value], list_a[i] = list_a[i], list_a[min_value]

    return list_a
print(selection_sort([4, 6, 2, 8, 7]))
```

Figure 3.2.7.2 Python code for Insertion sort considering a single array

### Implementation for larger arrays

To test the performance of Selection Sort on larger datasets, arrays of sizes 1000, 5000, 10000, 20000, 50000, and 100000 were generated. These arrays were kept consistent across all  $O(n^2)$  sorting algorithms to ensure a fair comparison. The sorting function was executed on each array, and the time taken was recorded. To improve accuracy, execution times were averaged over multiple runs. This approach allowed for an effective analysis of Selection Sort's scalability and efficiency on larger datasets.

## Advantages

- Selection Sort is straightforward and does not require complex logic, making it easy to implement and understand.
- It works efficiently for small lists where the overhead of more advanced sorting algorithms is unnecessary.

## Disadvantages

- With a time complexity of  $O(n^2)$ , Selection Sort becomes extremely slow as the input size increases.
- Unlike Insertion Sort, Selection Sort does not take advantage of partially sorted data, meaning it performs the same number of comparisons regardless of input order.

### 3.2.8 Counting Sort

Counting sort is a non-comparison-based sorting algorithm that operates in  $O(n + K)$  time complexity, where  $K$  is the range of input values. It is particularly efficient for sorting integers within a limited range. The algorithm works by counting the occurrences of each element in the input array and storing them in a frequency array. Then, a prefix sum is applied to determine the correct positions of the elements in the sorted array. Since Counting Sort only works with discrete values and does not compare elements directly, it is classified as a stable sorting algorithm that maintains the relative order of equal elements. However, it requires additional memory proportional to  $K$ , making it less efficient for datasets with a large range of values.

#### Concept:

1. Create an auxiliary array (count array) to store the frequency of each element in the input array. The size of this array is determined by the maximum value in the input.
2. Modify the count array to store cumulative sums, indicating the position of each element in the sorted order.
3. Traverse the input array in reverse order to maintain stability. Place each element in its correct position in the output array using the prefix sum values from the count array.
4. Copy the sorted elements from the output array back to the original array if required.

Ex: - Take a single array => [1, 0, 3, 1, 3, 1]

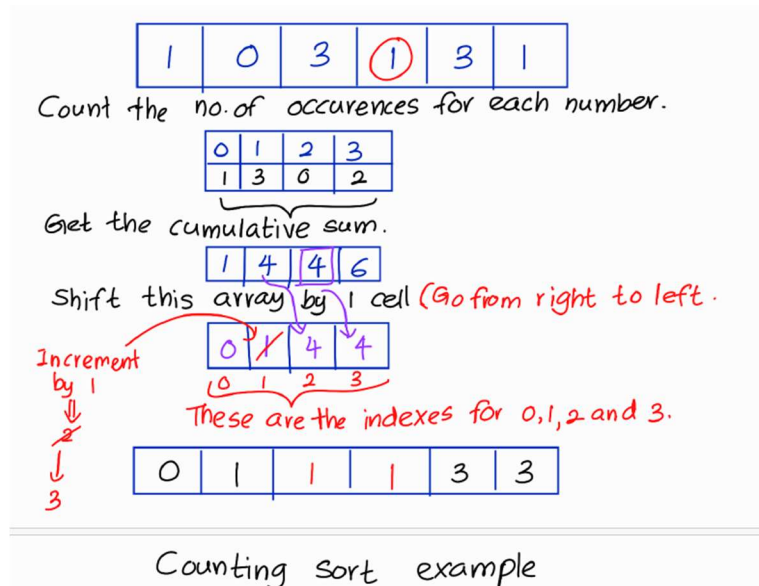


Figure 3.2.8.1 Counting Sort example for a single array

```
def counting_sort(l):
    size = len(l)
    output = [0] * size
    count = [0] * 10

    for i in range(0, size):
        count[l[i]] += 1

    for j in range(1, 10):
        count[j] += count[j-1]

    a = size-1
    while a >= 0:
        output[count[l[a]]-1] = l[a]
        count[l[a]] -= 1
        a -= 1

    for k in range(0, size):
        l[k] = output[k]

    l1 = [1,0,3,1,3,1]
    counting_sort(l1)
    print(l1)
```

Figure 3.2.8.2 Python code for Counting sort considering a single array

### **Implementation for larger arrays**

For larger arrays, this was done by generating three distinct random arrays of sizes 1000, 5000, 10000, 20000, 50000, and 100000. These arrays were kept consistent across all sorting algorithms to ensure fair and reliable comparison. The same arrays were then used to perform counting sort, with the input sizes increasing as described. By modifying the array parameters as the size increased, the algorithm's performance was tested across different array sizes, ensuring accuracy and a comprehensive evaluation of its behavior on larger datasets.

### **Advantages**

- Counting sort performs exceptionally well when the range of input values ( $K$ ) is small relative to the number of elements ( $n$ ). It can achieve linear time complexity  $O(n + K)$ , which makes it much faster than comparison-based algorithms for smaller ranges.
- Counting sort is a stable sorting algorithm, meaning that the relative order of elements with equal keys is preserved. This feature is beneficial when sorting elements with multiple attributes.

### **Disadvantages**

- Counting sort requires additional space proportional to the range of input values ( $K$ ), which can be inefficient when  $K$  is large. This makes the algorithm impractical for sorting large numbers with a vast range of possible values.
- Counting sort is primarily designed for integer data. Sorting floating-point numbers or strings, for example, requires additional modifications, making it less versatile compared to other algorithms like quicksort or mergesort.

### 3.2.9 Radix Sort

Radix sort is a non-comparative sorting algorithm that sorts a list of integers by processing individual digits. The algorithm works by sorting the elements starting from the least significant digit (the rightmost digit). This is done using a stable sorting algorithm, such as counting sort, to arrange the numbers based on the current digit. After sorting by the least significant digit, the algorithm moves to the next digit, and the process is repeated for all digits, from right to left, until the entire list is sorted.

The number of passes required in Radix sort depends on the number of digits in the largest number. For example, if the largest number has  $d$  digits, the algorithm will require  $d$  passes to sort the list. Since the time complexity of each pass is linear,  $O(n)$ , and there are  $d$  passes, the overall time complexity is  $O(n * d)$ , which is efficient when  $d$  (the number of digits) is relatively small compared to  $n$  (the number of elements). This makes Radix sort suitable for sorting large datasets of integers with a relatively small range of values.

Ex: - Take a single array  $\Rightarrow [53, 89, 150, 36, 633, 233]$

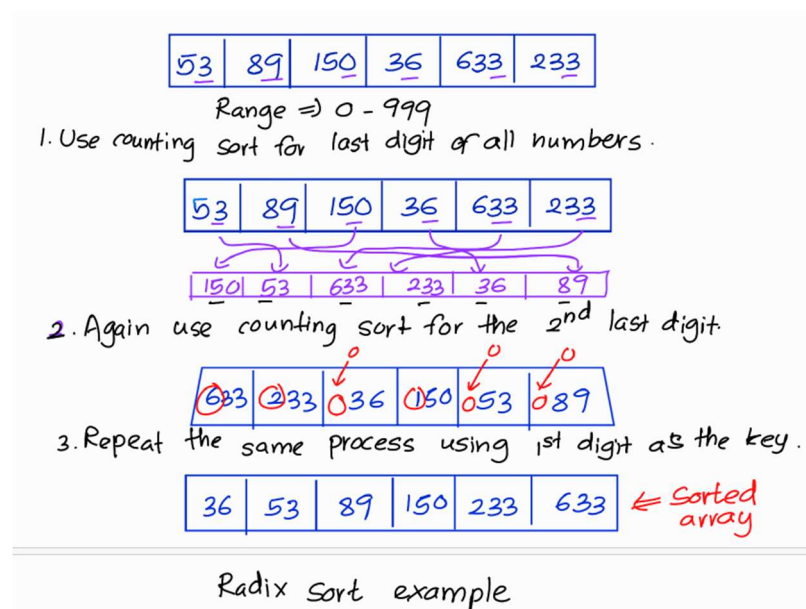


Figure 3.2.9.1 Radix Sort example for a single array

---

```

def counting_sort(l, place):
    size = len(l)
    output = [0] * size
    count = [0] * 10

    for i in range(0, size):
        index = l[i] // place
        count[index % 10] += 1
    for j in range(1, 10):
        count[j] += count[j-1]

    k = size-1
    while k >= 0:
        index = l[k] // place
        output[count[index % 10] - 1] = l[k]
        count[index % 10] -= 1
        k -= 1
    for a in range(0, size):
        l[a] = output[a]

def radix_sort(l):
    m = max(l)
    place = 1
    while m // place > 0:
        counting_sort(l, place)
        place *= 10

l1 = [53, 89, 150, 36, 633, 233]
radix_sort(l1)
print(l1)

```

*Figure 3.2.9.2 Python code for Radix sort considering a single array*

### Implementation for larger arrays

For larger arrays, Radix Sort was implemented by modifying the parameters for different input sizes (1000, 5000, 10000, 20000, 50000, and 100000) while maintaining the same basic structure of the algorithm. The code was executed on these larger arrays, and the same stable sorting technique (Counting Sort) was used for sorting based on each digit.

To handle larger arrays efficiently, the algorithm was optimized by adjusting the sorting passes based on the number of digits in the largest element within the array. The sorting was performed iteratively for each digit, starting from the least significant digit and moving towards the most significant digit. By repeating this process for all digit positions, the algorithm was able to handle large datasets effectively while keeping the sorting time complexity linear with respect to the number of elements and the number of digits.

Additionally, the arrays were generated randomly to ensure consistency, and time measurements were recorded for different input sizes to observe the algorithm's performance on larger datasets.

## Advantages

- Radix Sort performs efficiently when dealing with large datasets of numbers with a fixed number of digits, achieving linear time complexity of  $O(n)$ , making it faster than comparison-based sorting algorithms for large arrays.
- Radix Sort is a stable sorting algorithm, meaning that elements with equal values maintain their relative order from the original input array. This property can be important in specific use cases where stability is required.

## Disadvantages

- Radix Sort requires additional space for the counting sort used as a stable sorting method. For large arrays with wide ranges of values, this can lead to high space consumption.
- Radix Sort is mainly suitable for sorting integers or fixed-length data, and is not directly applicable to floating-point numbers or data types that don't have a natural ordering of digits, limiting its versatility.

### 3.2.10 Bucket Sort

Bucket Sort is a distribution-based sorting algorithm that works by dividing the input array into a number of smaller, equally sized subarrays called buckets. Each bucket is then sorted individually, either using a different sorting algorithm or recursively applying the same bucket sort technique. After sorting the elements within each bucket, the algorithm gathers them back into a single sorted array.

The concept of Bucket Sort can be viewed as a scatter-order-gather approach. First, the elements are scattered across buckets based on their values, then each bucket is ordered independently, and finally, the gathering step reassembles the sorted elements back into the original array. This approach is highly efficient when the input elements are uniformly distributed over a range.

Ex: - Take a single array  $\Rightarrow$  [56, 34, 89, 45, 23, 65, 33, 12, 9, 48, 72, 96]

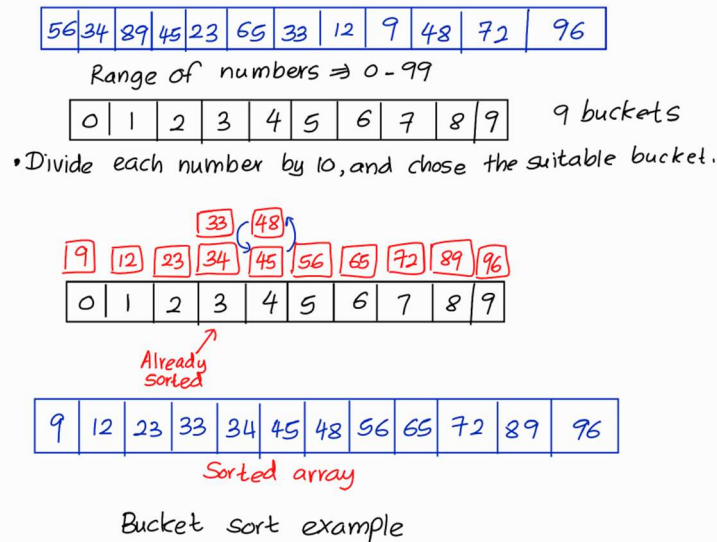


Figure 3.2.10.1 Bucket sort example for a single array

```
def bucketSort(array):
    max_value = max(array)

    bucket = [[] for _ in range(len(array))]

    for j in array:
        # Normalize and find the index
        index_b = int(len(array) * j / max_value) if max_value != 0 else 0

        index_b = min(index_b, len(array) - 1)

        bucket[index_b].append(j)

    # Sort each bucket
    for i in range(len(array)):
        bucket[i] = sorted(bucket[i])

    k = 0
    for i in range(len(array)):
        for j in range(len(bucket[i])):
            array[k] = bucket[i][j]
            k += 1

    return array

# Example usage
l1 = [56, 34, 89, 45, 23, 65, 33, 12, 9, 48, 72, 96]
res = bucketSort(l1)
print(res)
```

Figure 3.2.10.2 Python code for Bucket sort considering a single array

## Advantages

- When the input data is uniformly distributed, Bucket Sort can be extremely efficient, with a time complexity of  $O(n + k)$ , where  $n$  is the number of elements and  $k$  is the number of buckets.
- Since each bucket is sorted independently, the sorting of the buckets can be easily parallelized, making it suitable for distributed or multi-core environments, potentially reducing sorting time further.

## Disadvantages

- Bucket Sort requires additional memory for the buckets, and in cases of large input data with a high range of values, this can lead to significant space overhead.
- Bucket Sort is not universally applicable, as it works best with numeric data and specific types of distributions. For data that isn't uniformly distributed or when there is a wide variance in values, the algorithm's efficiency can degrade.

## 3.3 Analysis of various Sorting Algorithms

### 3.3.1 Big O notation

Big O notation describes how an algorithm's performance changes as the input size grows. As more inputs or data points are added, two aspects of complexity may increase:

1. **Time Complexity** – The time required to execute the algorithm.
2. **Space Complexity** – The memory required to store data during execution.

The variable '**n**' represents the number of inputs. If **n scales up to infinity**, the rate at which complexity increases determines the efficiency of an algorithm.

### Common Complexity Classes:

- **$O(n)$  – Linear Complexity:**

Each input corresponds to one step in execution. If looping over each element takes exactly one year, doubling the inputs would take two years.

- **$O(\log n)$  – Logarithmic Complexity:**

A **binary search** operates in logarithmic time. Instead of checking each element one by one, it **divides the search space in half** at each step, making it much more efficient for large datasets.

- **$O(1)$  – Constant Complexity:**

Some operations, like accessing an element by its index in an array, always take the same amount of time, regardless of the array size.

- **$O(n^2)$  – Quadratic Complexity:**

A **nested loop** (a loop within a loop) results in the number of operations growing exponentially with inputs.

- **$O(2^n)$  – Exponential Complexity:**

Some algorithms, such as those that generate all possible subsets of an array, experience extreme growth in complexity, making them highly inefficient for large inputs.

Some algorithms have multiple growth rates, but **Big O notation focuses on the worst-case scenario** to determine efficiency. Always the most optimized approach is aimed at when designing an algorithm.

### 3.3.2 Time Complexity and Space Complexity of Various Sorting Algorithms

The comparison of various search algorithms is done based on their time and space complexity is shown below.

| No. | Sorting Algorithm | Time Complexity |               |               | Space Complexity | Stable |
|-----|-------------------|-----------------|---------------|---------------|------------------|--------|
|     |                   | Best Case       | Average Case  | Worst Case    |                  |        |
| 1)  | Quick sort        | $O(n \log n)$   | $O(n \log n)$ | $O(n^2)$      | $O(1)$           | No     |
| 2)  | Merge sort        | $O(n \log n)$   | $O(n \log n)$ | $O(n \log n)$ | $O(n)$           | Yes    |
| 3)  | Heap sort         | $O(n \log n)$   | $O(n \log n)$ | $O(n \log n)$ | $O(1)$           | No     |
| 4)  | Insertion sort    | $O(n)$          | $O(n^2)$      | $O(n^2)$      | $O(1)$           | Yes    |
| 5)  | Bubble sort       | $O(n)$          | $O(n^2)$      | $O(n^2)$      | $O(1)$           | Yes    |
| 6)  | Selection sort    | $O(n^2)$        | $O(n^2)$      | $O(n^2)$      | $O(1)$           | Yes    |
| 7)  | Counting sort     | $O(n + k)$      | $O(n + k)$    | $O(n + k)$    | $O(k)$           | Yes    |
| 8)  | Bucket sort       | $O(n + k)$      | $O(n)$        | $O(n^2)$      | $O(n + k)$       | Yes    |
| 9)  | Radix sort        | $O(nk)$         | $O(nk)$       | $O(nk)$       | $O(n + d)$       | Yes    |

*Table 1: Analysis of various sorting algorithms*

## 4 RESULTS & DISCUSSION

### 4.1 Experimental Analysis

#### 4.1.1 Case 1: Running Time of $O(n \log n)$ sorting algorithms

Results of Case 1 are shown in Table 2. It shows the running time of  $O(n \log n)$  sorting algorithms, the size of array varied from 1000 to 100000.

| Number of elements | Quicksort (s) | Randomized Quicksort (s) | Merge sort (s) | Heap sort (s) |
|--------------------|---------------|--------------------------|----------------|---------------|
| 1000               | 0.001718      | 0.001528                 | 0.002507       | 0.005174      |
| 5000               | 0.012086      | 0.007255                 | 0.014095       | 0.029454      |
| 10000              | 0.014255      | 0.017730                 | 0.023345       | 0.058815      |
| 20000              | 0.036823      | 0.039207                 | 0.050295       | 0.151036      |
| 50000              | 0.094034      | 0.112441                 | 0.141380       | 0.355688      |
| 100000             | 0.195078      | 0.203781                 | 0.297685       | 0.795295      |

Table 2: Comparison of  $O(n \log n)$  sorting algorithms

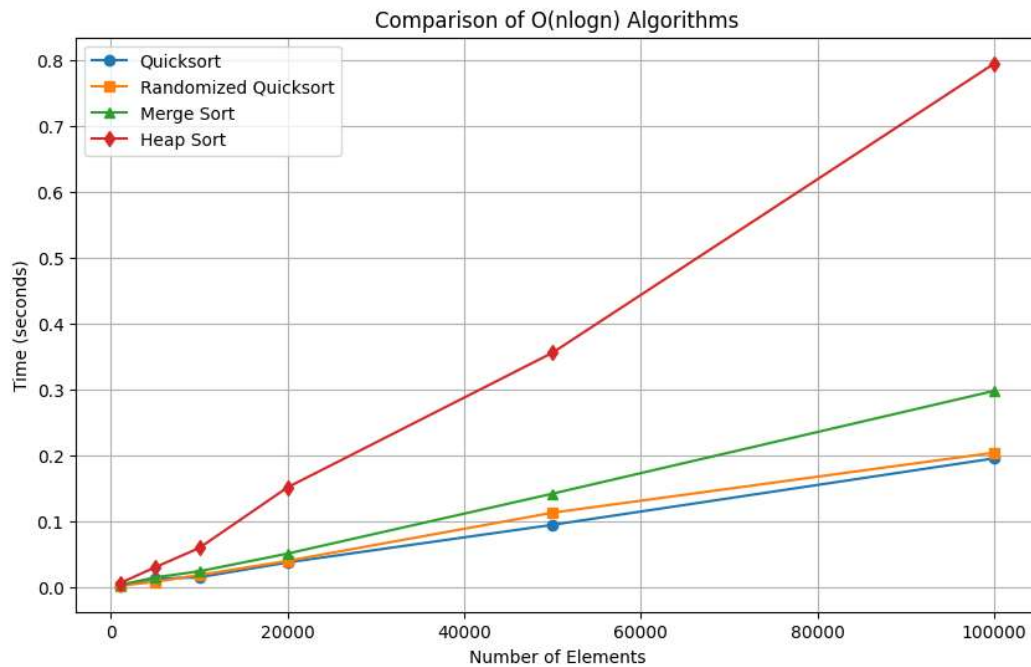


Figure 4.1.1.1 Graph of comparison of  $O(n \log n)$  sorting algorithms

#### 4.1.2 Case 2: Running Time of $O(n^2)$ sorting algorithms

Results of Case 2 are shown in Table 3. It shows the running time of  $O(n^2)$  sorting algorithms, the size of array varied from 1000 to 100000.

| Number of elements | Bubble sort (s) | Optimized bubble sort (s) | Insertion sort (s) | Selection sort (s) |
|--------------------|-----------------|---------------------------|--------------------|--------------------|
| 1000               | 0.055383        | 0.042258                  | 0.015134           | 0.023219           |
| 5000               | 1.772399        | 0.986894                  | 0.442173           | 0.380653           |
| 10000              | 7.334010        | 4.209195                  | 1.661074           | 1.571825           |
| 20000              | 31.267234       | 17.191364                 | 7.101128           | 6.301917           |
| 50000              | 220.156126      | 114.085153                | 43.808798          | 42.143270          |
| 100000             | 994.317709      | 575.297935                | 196.433718         | 173.437922         |

Table 3: Comparison of  $O(n^2)$  sorting algorithms

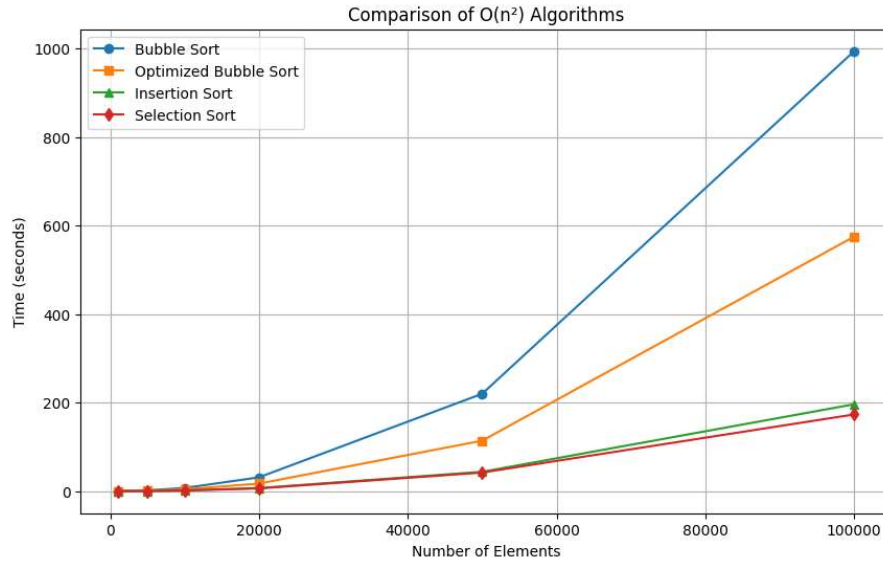


Figure 4.1.2.1 Graph of comparison of  $O(n^2)$  sorting algorithms

#### 4.1.3 Case 3: Running Time of non-comparison-based sorting algorithms

Results of Case 3 are shown in Table 4. It shows the running time of non-comparison-based sorting algorithms, the size of array varied from 1000 to 100000.

| Number of elements | Counting sort (s) | Bucket sort (s) | Radix sort (s) |
|--------------------|-------------------|-----------------|----------------|
| 1000               | 0.000180          | 0.003000        | 0.001000       |
| 5000               | 0.001304          | 0.003489        | 0.004967       |
| 10000              | 0.004759          | 0.004864        | 0.011195       |
| 20000              | 0.005966          | 0.018550        | 0.025977       |
| 50000              | 0.015832          | 0.038282        | 0.068532       |
| 100000             | 0.045486          | 0.102539        | 0.175246       |

Table 4: Comparison of non-comparison-based sorting algorithms

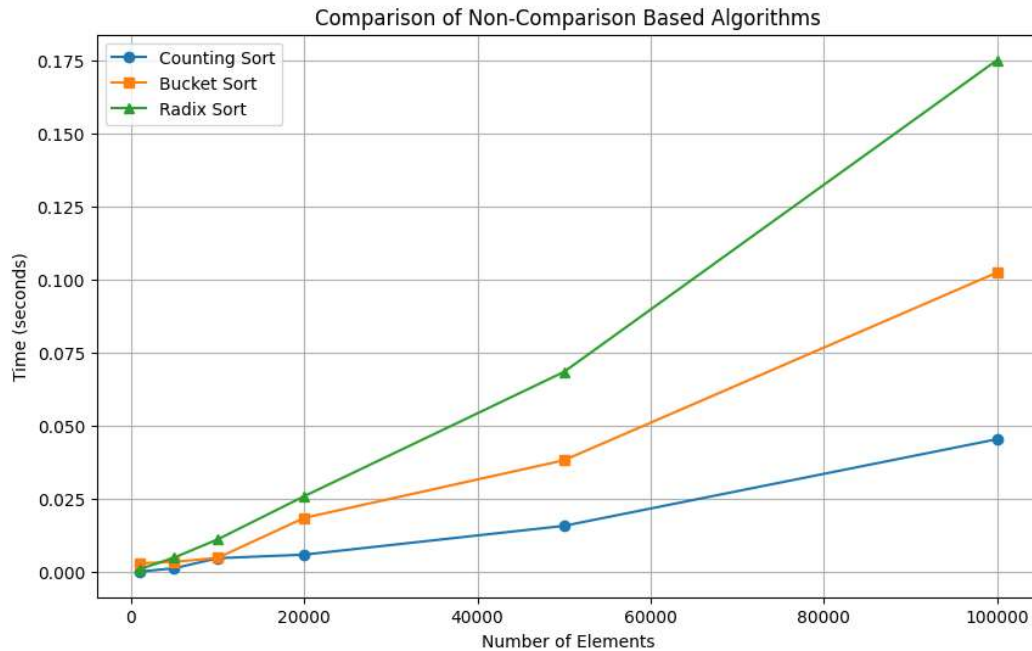


Figure 4.1.3.1 Graph of comparison of non-comparison-based sorting algorithms

## 4.2 Discussion

### 4.2.1 Comparison of $O(n \log n)$ sorting algorithms

The results from Table 2 show the execution times for Quicksort, Randomized Quicksort, Merge Sort, and Heap Sort across different input sizes. As expected, all these algorithms follow  $O(n \log n)$  time complexity, which is significantly better than  $O(n^2)$  sorting algorithms for large datasets.

- Quicksort and Randomized Quicksort:** The standard Quicksort performed well, but the **randomized version consistently showed slightly better performance**, especially as the input size increased. This aligns with the theoretical expectation that choosing a random pivot **reduces the likelihood of worst-case  $O(n^2)$  performance**, ensuring a more balanced partitioning.
- Merge Sort:** The performance of Merge Sort was close to Quicksort but slightly slower due to its **additional memory usage and recursive nature**. However, unlike Quicksort, Merge Sort maintains **consistent  $O(n \log n)$  complexity even in the worst case**, making it a reliable option for large datasets.

- **Heap Sort:** Among the  $O(n \log n)$  algorithms, Heap Sort had the **slowest execution times**. This can be attributed to the **constant factors** involved in maintaining the heap structure and frequent swaps, which introduce additional overhead.

As the input size increased, **Quicksort and Randomized Quicksort remained the fastest** due to their in-place sorting and efficient partitioning strategy. The use of the same randomly generated arrays across all sorting algorithms ensured fair testing, eliminating discrepancies caused by variations in input data.

#### 4.2.2 Comparison of $O(n^2)$ sorting algorithms

The results from Table 3 confirm that  **$O(n^2)$  sorting algorithms perform poorly on large datasets**, as their execution times increased significantly with input size.

- **Bubble Sort and Optimized Bubble Sort:** The standard **Bubble Sort was the slowest** due to its repeated element swapping, requiring  $O(n^2)$  comparisons and swaps in the worst case. The **optimized version** showed improvement by reducing unnecessary passes when the array became sorted early, but it still remained inefficient for larger datasets.
- **Insertion Sort:** Among the  $O(n^2)$  algorithms, **Insertion Sort performed the best for smaller input sizes** because it is adaptive, meaning that it performs well when the array is partially sorted. However, for larger datasets, its performance degraded as the number of comparisons and shifts increased.
- **Selection Sort:** Selection Sort had execution times similar to Insertion Sort but was slightly **faster** in this experiment, likely due to its **fewer swaps and better memory access patterns**. Unlike Insertion Sort, which shifts elements multiple times, Selection Sort performs **a fixed number of swaps**, reducing the overall data movement. However, since it does not benefit from partially sorted arrays, its efficiency remains  **$O(n^2)$  in all cases**, making it impractical for very large datasets.

In theory, **Insertion Sort should outperform Selection Sort for small input sizes**, but as the input grows, both have  **$O(n^2)$  complexity**, making them inefficient. However, the experimental results suggest that **Selection Sort benefited from fewer swaps and better memory handling**, which made it slightly faster than Insertion Sort for larger datasets in this case.

As expected, all  $O(n^2)$  sorting algorithms performed much worse than  $O(n \log n)$  algorithms for larger datasets. The use of **the same arrays across all  $O(n^2)$  algorithms** ensured consistency in comparison.

#### 4.2.3 Comparison of Non-Comparison-Based Sorting Algorithms

The results from Table 4 highlight that **non-comparison-based sorting algorithms were the most efficient** when applied to large datasets.

- **Counting Sort:** This was the **fastest algorithm** due to its  $O(n + k)$  complexity, which avoids element-wise comparisons. However, it requires additional memory proportional to the range of input values ( $k$ ), making it impractical for datasets with a large value range.
- **Radix Sort:** The performance of Radix Sort remained close to Counting Sort, as expected, since it uses Counting Sort as a subroutine. While Radix Sort was slightly slower, it is more **space efficient than Counting Sort** when sorting large integers.
- **Bucket Sort:** The execution times of Bucket Sort varied depending on the input distribution. It performed efficiently for **uniformly distributed** data but took longer when the distribution was uneven, as some buckets contained significantly more elements than others.

Among these algorithms, **Counting Sort was the fastest, but its practicality depends on the range of input values**. Radix Sort provides a balanced alternative, while Bucket Sort is beneficial when dealing with uniformly distributed floating-point values. Using the **same randomly generated arrays** for these algorithms ensured fair testing.

## 5 CONCLUSION

The comparative analysis of various sorting algorithms provided insights into their performance across different input sizes and categories. The study included comparison-based sorting algorithms such as Quicksort, Merge Sort, Heap Sort, Bubble Sort, Insertion Sort, and Selection Sort, as well as non-comparison-based sorting algorithms like Counting Sort, Radix Sort, and Bucket Sort. Execution times were recorded and analyzed for datasets of increasing sizes (1000 to 100,000 elements) to evaluate their efficiency.

The results showed that  $O(n \log n)$  algorithms consistently outperformed  $O(n^2)$  algorithms, as expected. Among them, Quicksort and Randomized Quicksort were the fastest, with the randomized version performing slightly better by reducing the likelihood of worst-case  $O(n^2)$  complexity. Merge Sort demonstrated stable performance across all input sizes but was slightly slower due to its additional memory usage. Heap Sort, while maintaining the same theoretical time complexity, had the slowest execution times among the  $O(n \log n)$  algorithms due to frequent swaps and heap maintenance overhead.

For  $O(n^2)$  algorithms, execution times increased significantly with larger datasets, reinforcing their inefficiency for large-scale sorting. Insertion Sort performed the best for small inputs, benefiting from its adaptability to nearly sorted arrays. Selection Sort unexpectedly outperformed Insertion Sort for larger input sizes, likely due to its fewer swaps and better memory access patterns. Bubble Sort was the slowest, with the optimized version improving performance slightly but still remaining inefficient for large datasets.

Among the non-comparison-based sorting algorithms, Counting Sort was the fastest overall, achieving near  $O(n)$  performance due to its direct indexing method. However, its high memory consumption made it impractical for datasets with a large value range. Radix Sort closely followed Counting Sort, offering a balance between efficiency and memory usage, making it a more practical option for large datasets. Bucket Sort performed well when the data was uniformly distributed but showed inconsistent results when the distribution was uneven, making it less predictable compared to Counting and Radix Sort.

Using the same datasets across algorithms ensured fair comparisons by eliminating input randomness as a variable factor. This approach allowed for a more accurate evaluation of each algorithm's efficiency and behavior under identical conditions, ensuring that differences in performance were solely due to algorithmic efficiency rather than variations in input data.

From the overall analysis, Randomized Quicksort emerged as the best general-purpose sorting algorithm, offering both speed and minimal memory usage. Merge Sort is preferable when stability is required, while Heap Sort may be useful in scenarios where in-place sorting is necessary but not time-sensitive. For small or nearly sorted datasets, Insertion Sort is the best choice, while Selection Sort may be preferable in cases where minimizing swaps is important. Among non-comparison-based sorting algorithms, Counting Sort and Radix Sort are the most efficient for sorting large datasets when the range of values is limited.

Overall, the experimental results aligned well with theoretical expectations, with some deviations, such as Selection Sort outperforming Insertion Sort for larger datasets and Quicksort performing better than Merge Sort in terms of execution time. The findings highlight the importance of selecting the appropriate sorting algorithm based on factors such as dataset size, distribution, memory constraints, and performance requirements.

## APPENDICES – PROGRAM CODE

### Comparison of $O(n \log n)$ algorithms

#### Implementation of Quicksort for larger arrays

```
#Quicksort algorithm implementation
#Arraysize = 1000
import random
import time
import copy # To make copies of the arrays

# Partition function
def partition(arr, low, high):
    pivot_index = random.randint(low, high) # Select a random pivot
    pivot = arr[pivot_index]

    arr[pivot_index], arr[high] = arr[high], arr[pivot_index] # Swap pivot with last element

    i = low - 1 # Pointer for the smaller element
    for j in range(low, high):
        if arr[j] <= pivot:
            i += 1
            arr[i], arr[j] = arr[j], arr[i]
    arr[i + 1], arr[high] = arr[high], arr[i + 1]
    return i + 1

# Quick Sort function
def randomizedQuickSort(arr, low, high):
    if low < high:
        pivot_index = partition(arr, low, high)
        randomizedQuickSort(arr, low, pivot_index - 1)
        randomizedQuickSort(arr, pivot_index + 1, high)

# ----- GENERATING ARRAYS -----
num_arrays = 3
array_size = 1000
original_quick_sort_arrays_1000 = [[random.randint(1, 1000) for _ in range(array_size)] for _ in range(num_arrays)]

# ----- QUICK SORT TIMING -----
quick_sort_times = []

for index, original_array in enumerate(original_quick_sort_arrays_1000, 1):
    arr_copy = copy.deepcopy(original_array) # Copy to preserve the original array

    start_time = time.time()
    randomizedQuickSort(arr_copy, 0, len(arr_copy) - 1)
    end_time = time.time()

    time_taken = end_time - start_time
    quick_sort_times.append(time_taken)
    print(f"Time taken for quick sort on array {index}: {time_taken:.6f} seconds")

# Calculate and print average quick sort time
avg_quick_sort_time = sum(quick_sort_times) / len(quick_sort_times)
print(f"\nAverage time taken for quick sort: {avg_quick_sort_time:.6f} seconds")
```

Array size = 1000

```

#Arraysize = 100000
import random
import time
import copy # To make copies of the arrays

# Partition function
def partition(arr, low, high):
    pivot_index = random.randint(low, high) # Select a random pivot
    pivot = arr[pivot_index]

    arr[pivot_index], arr[high] = arr[high], arr[pivot_index] # Swap pivot with last element

    i = low - 1 # Pointer for the smaller element
    for j in range(low, high):
        if arr[j] <= pivot:
            i += 1
            arr[i], arr[j] = arr[j], arr[i]
    arr[i + 1], arr[high] = arr[high], arr[i + 1]
    return i + 1

# Quick Sort function
def randomizedQuickSort(arr, low, high):
    if low < high:
        pivot_index = partition(arr, low, high)
        randomizedQuickSort(arr, low, pivot_index - 1)
        randomizedQuickSort(arr, pivot_index + 1, high)

# ----- GENERATING ARRAYS -----
num_arrays = 3
array_size = 100000
original_quick_sort_arrays_100000 = [[random.randint(1, 100000) for _ in range(array_size)] for _ in range(num_arrays)]

# ----- QUICK SORT TIMING -----
quick_sort_times = []

for index, original_array in enumerate(original_quick_sort_arrays_100000, 1):
    arr_copy = copy.deepcopy(original_array) # Copy to preserve the original array

    start_time = time.time()
    randomizedQuickSort(arr_copy, 0, len(arr_copy) - 1)
    end_time = time.time()

    time_taken = end_time - start_time
    quick_sort_times.append(time_taken)
    print(f"Time taken for quick sort on array {index}: {time_taken:.6f} seconds")

# Calculate and print average quick sort time
avg_quick_sort_time = sum(quick_sort_times) / len(quick_sort_times)
print(f"\nAverage time taken for quick sort: {avg_quick_sort_time:.6f} seconds")

```

Array size = 100000

## Implementation of Randomized Quicksort for larger arrays

```
#Randomized quicksort implementation
#Arraysize = 1000
import random
import time
import copy # To make copies of the arrays

# Partition function for Randomized Quick Sort
def partition(arr, low, high):
    pivot_index = random.randint(low, high)
    pivot = arr[pivot_index]

    arr[pivot_index], arr[high] = arr[high], arr[pivot_index]

    i = low - 1
    for j in range(low, high):
        if arr[j] <= pivot:
            i += 1
            arr[i], arr[j] = arr[j], arr[i]
    arr[i + 1], arr[high] = arr[high], arr[i + 1]
    return i + 1

# Randomized Quick Sort function
def randomizedQuickSort(arr, low, high):
    if low < high:
        pivot_index = partition(arr, low, high)
        randomizedQuickSort(arr, low, pivot_index - 1)
        randomizedQuickSort(arr, pivot_index + 1, high)

# ----- RANDOMIZED QUICK SORT TIMING -----
randomized_quick_sort_times = []

for index, original_array in enumerate(original_quick_sort_arrays_1000, 1):
    arr_copy = copy.deepcopy(original_array) # Copy to preserve the original array

    start_time = time.time()
    randomizedQuickSort(arr_copy, 0, len(arr_copy) - 1)
    end_time = time.time()

    time_taken = end_time - start_time
    randomized_quick_sort_times.append(time_taken)
    print(f"Time taken for randomized quick sort on array {index}: {time_taken:.6f} seconds")

# Calculate and print average randomized quick sort time
avg_randomized_quick_sort_time = sum(randomized_quick_sort_times) / len(randomized_quick_sort_times)
print(f"\nAverage time taken for randomized quick sort: {avg_randomized_quick_sort_time:.6f} seconds")
```

Array size = 1000

```

#Arraysize = 100000
import random
import time
import copy # To make copies of the arrays

# Partition function for Randomized Quick Sort
def partition(arr, low, high):
    pivot_index = random.randint(low, high)
    pivot = arr[pivot_index]

    arr[pivot_index], arr[high] = arr[high], arr[pivot_index]

    i = low - 1
    for j in range(low, high):
        if arr[j] <= pivot:
            i += 1
            arr[i], arr[j] = arr[j], arr[i]
    arr[i + 1], arr[high] = arr[high], arr[i + 1]
    return i + 1

# Randomized Quick Sort function
def randomizedQuickSort(arr, low, high):
    if low < high:
        pivot_index = partition(arr, low, high)
        randomizedQuickSort(arr, low, pivot_index - 1)
        randomizedQuickSort(arr, pivot_index + 1, high)

# ----- RANDOMIZED QUICK SORT TIMING -----
randomized_quick_sort_times = []

for index, original_array in enumerate(original_quick_sort_arrays_100000, 1):
    arr_copy = copy.deepcopy(original_array) # Copy to preserve the original array

    start_time = time.time()
    randomizedQuickSort(arr_copy, 0, len(arr_copy) - 1)
    end_time = time.time()

    time_taken = end_time - start_time
    randomized_quick_sort_times.append(time_taken)
    print(f"Time taken for randomized quick sort on array {index}: {time_taken:.6f} seconds")

# Calculate and print average randomized quick sort time
avg_randomized_quick_sort_time = sum(randomized_quick_sort_times) / len(randomized_quick_sort_times)
print(f"\nAverage time taken for randomized quick sort: {avg_randomized_quick_sort_time:.6f} seconds")

```

Array size = 100000

## Implementation of Merge Sort for larger arrays

```

#Mergesort implementation
#Arraysize = 1000
import time
import copy # To make copies of the arrays

# Merge function
def merge(arr, left, mid, right):
    left_part = arr[left:mid + 1] # Left subarray
    right_part = arr[mid + 1:right + 1] # Right subarray

    i, j, k = 0, 0, left # Initialize pointers

    while i < len(left_part) and j < len(right_part):
        if left_part[i] <= right_part[j]:
            arr[k] = left_part[i]
            i += 1
        else:
            arr[k] = right_part[j]
            j += 1
        k += 1

    # Copy remaining elements (if any)
    while i < len(left_part):
        arr[k] = left_part[i]
        i += 1
        k += 1

```

```

        while j < len(right_part):
            arr[k] = right_part[j]
            j += 1
            k += 1

# Merge Sort function
def mergeSort(arr, left, right):
    if left < right:
        mid = (left + right) // 2
        mergeSort(arr, left, mid)
        mergeSort(arr, mid + 1, right)
        merge(arr, left, mid, right)

```

```

# ----- MERGE SORT TIMING -----
merge_sort_times = []

for index, original_array in enumerate(original_quick_sort_arrays_1000, 1): # Using the same 3 arrays from Quick Sort
    arr_copy = copy.deepcopy(original_array) # Copy to preserve the original array

    start_time = time.time()
    mergeSort(arr_copy, 0, len(arr_copy) - 1)
    end_time = time.time()

    time_taken = end_time - start_time
    merge_sort_times.append(time_taken)
    print(f"Time taken for merge sort on array {index}: {time_taken:.6f} seconds")

# Calculate and print average merge sort time
avg_merge_sort_time = sum(merge_sort_times) / len(merge_sort_times)
print(f"\nAverage time taken for merge sort: {avg_merge_sort_time:.6f} seconds")

```

Array size = 1000

```

#Arraysize = 100000
import time
import copy # To make copies of the arrays

# Merge function
def merge(arr, left, mid, right):
    left_part = arr[left:mid + 1] # Left subarray
    right_part = arr[mid + 1:right + 1] # Right subarray

    i, j, k = 0, 0, left # Initialize pointers

    while i < len(left_part) and j < len(right_part):
        if left_part[i] <= right_part[j]:
            arr[k] = left_part[i]
            i += 1
        else:
            arr[k] = right_part[j]
            j += 1
        k += 1

    # Copy remaining elements (if any)
    while i < len(left_part):
        arr[k] = left_part[i]
        i += 1
        k += 1

```

```

        while j < len(right_part):
            arr[k] = right_part[j]
            j += 1
            k += 1

# Merge Sort function
def mergeSort(arr, left, right):
    if left < right:
        mid = (left + right) // 2
        mergeSort(arr, left, mid)
        mergeSort(arr, mid + 1, right)
        merge(arr, left, mid, right)

```

```

# ----- MERGE SORT TIMING -----
merge_sort_times = []

for index, original_array in enumerate(original_quick_sort_arrays_100000, 1): # Using the same 3 arrays from Quick Sort
    arr_copy = copy.deepcopy(original_array) # Copy to preserve the original array

    start_time = time.time()
    mergeSort(arr_copy, 0, len(arr_copy) - 1)
    end_time = time.time()

    time_taken = end_time - start_time
    merge_sort_times.append(time_taken)
    print(f"Time taken for merge sort on array {index}: {time_taken:.6f} seconds")

# Calculate and print average merge sort time
avg_merge_sort_time = sum(merge_sort_times) / len(merge_sort_times)
print(f"\nAverage time taken for merge sort: {avg_merge_sort_time:.6f} seconds")

```

Array size = 100000

## Implementation of Heap Sort for larger arrays

```

#Heapsort implementation
#Arraysize = 1000
import time
import copy # To make copies of the arrays

# Swap function
def swap(lst, i, j):
    lst[i], lst[j] = lst[j], lst[i]

# Sift down function for heapify
def siftDown(lst, i, upper):
    while True:
        l, r = 2 * i + 1, 2 * i + 2 # Left and right child indices

        if max(l, r) < upper:
            if lst[i] >= max(lst[l], lst[r]):
                break
            elif lst[l] > lst[r]:
                swap(lst, i, l)
                i = l
            else:
                swap(lst, i, r)
                i = r

```

```

        elif l < upper:
            if lst[l] > lst[i]:
                swap(lst, i, l)
                i = l
            else:
                break
        elif r < upper:
            if lst[r] > lst[i]:
                swap(lst, i, r)
                i = r
            else:
                break
        else:
            break

# Heap Sort function
def heapsort(lst):
    # Build max heap
    for j in range((len(lst) - 2) // 2, -1, -1):
        siftDown(lst, j, len(lst))

    # Extract elements from heap
    for end in range(len(lst) - 1, 0, -1):
        swap(lst, 0, end)
        siftDown(lst, 0, end)

```

```

# ----- HEAP SORT TIMING -----
heap_sort_times = []

# Using the same 3 arrays from QuickSort (original_quick_sort_arrays_1000)
for index, original_array in enumerate(original_quick_sort_arrays_1000, 1):
    arr_copy = copy.deepcopy(original_array) # Copy to preserve the original array

    start_time = time.time()
    heapsort(arr_copy)
    end_time = time.time()

    time_taken = end_time - start_time
    heap_sort_times.append(time_taken)
    print(f"Time taken for heap sort on array {index}: {time_taken:.6f} seconds")

# Calculate and print average heap sort time
avg_heap_sort_time = sum(heap_sort_times) / len(heap_sort_times)
print(f"\nAverage time taken for heap sort: {avg_heap_sort_time:.6f} seconds")

```

Array size = 1000

```

#Arraysize = 100000
import time
import copy # To make copies of the arrays

# Swap function
def swap(lst, i, j):
    lst[i], lst[j] = lst[j], lst[i]

# Sift down function for heapify
def siftDown(lst, i, upper):
    while True:
        l, r = 2 * i + 1, 2 * i + 2 # Left and right child indices

        if max(l, r) < upper:
            if lst[i] >= max(lst[l], lst[r]):
                break
            elif lst[l] > lst[r]:
                swap(lst, i, l)
                i = l
            else:
                swap(lst, i, r)
                i = r

```

```

        elif l < upper:
            if lst[l] > lst[i]:
                swap(lst, i, l)
                i = l
            else:
                break
        elif r < upper:
            if lst[r] > lst[i]:
                swap(lst, i, r)
                i = r
            else:
                break
        else:
            break

# Heap Sort function
def heapsort(lst):
    # Build max heap
    for j in range((len(lst) - 2) // 2, -1, -1):
        siftDown(lst, j, len(lst))

    # Extract elements from heap
    for end in range(len(lst) - 1, 0, -1):
        swap(lst, 0, end)
        siftDown(lst, 0, end)

```

```

# ----- HEAP SORT TIMING -----
heap_sort_times = []

# Using the same 3 arrays from QuickSort (original_quick_sort_arrays_100000)
for index, original_array in enumerate(original_quick_sort_arrays_100000, 1):
    arr_copy = copy.deepcopy(original_array) # Copy to preserve the original array

    start_time = time.time()
    heapsort(arr_copy)
    end_time = time.time()

    time_taken = end_time - start_time
    heap_sort_times.append(time_taken)
    print(f"Time taken for heap sort on array {index}: {time_taken:.6f} seconds")

# Calculate and print average heap sort time
avg_heap_sort_time = sum(heap_sort_times) / len(heap_sort_times)
print(f"\nAverage time taken for heap sort: {avg_heap_sort_time:.6f} seconds")

```

Array size = 100000

## Comparison of $O(n^2)$ algorithms

### Implementation of Bubble Sort for larger arrays

```
#Bubble sort implementation
#Array size = 1000
import random
import time
import copy # To make copies of the arrays

# Generate 3 new random arrays of size 1000 (stored in original_bubble_arrays)
num_arrays = 3
array_size = 1000
original_bubble_arrays_1000 = [[random.randint(1, 1000) for _ in range(array_size)] for _ in range(num_arrays)]

# Bubble Sort function
def bubble(list_a):
    indexing_length = len(list_a) - 1
    sorted = False # Create variable for sorted and set it equal to false

    while not sorted:
        sorted = True

        for i in range(0, indexing_length):
            if list_a[i] > list_a[i + 1]:
                sorted = False
                list_a[i], list_a[i + 1] = list_a[i + 1], list_a[i] # Switch these values

    return list_a

# ----- BUBBLE SORT TIMING -----
bubble_sort_times = []

for index, original_array in enumerate(original_bubble_arrays_1000, 1):
    arr_copy = copy.deepcopy(original_array) # Copy to preserve the original array

    start_time = time.time()
    bubble(arr_copy)
    end_time = time.time()

    time_taken = end_time - start_time
    bubble_sort_times.append(time_taken)
    print(f"Time taken for bubble sort on array {index}: {time_taken:.6f} seconds")

# Calculate and print average bubble sort time
avg_bubble_sort_time = sum(bubble_sort_times) / len(bubble_sort_times)
print(f"\nAverage time taken for bubble sort: {avg_bubble_sort_time:.6f} seconds")
```

Array size = 1000

```

#Array size = 100000
import random
import time
import copy # To make copies of the arrays

# Generate 3 new random arrays of size 100000 (stored in original_bubble_arrays)
num_arrays = 3
array_size = 100000
original_bubble_arrays_100000 = [[random.randint(1, 100000) for _ in range(array_size)] for _ in range(num_arrays)]

# Bubble Sort function
def bubble(list_a):
    indexing_length = len(list_a) - 1
    sorted = False

    while not sorted:
        sorted = True

        for i in range(0, indexing_length):
            if list_a[i] > list_a[i + 1]:
                sorted = False
                list_a[i], list_a[i + 1] = list_a[i + 1], list_a[i] # Switch these values

    return list_a

# ----- BUBBLE SORT TIMING -----
bubble_sort_times = []

for index, original_array in enumerate(original_bubble_arrays_100000, 1):
    arr_copy = copy.deepcopy(original_array) # Copy to preserve the original array

    start_time = time.time()
    bubble(arr_copy)
    end_time = time.time()

    time_taken = end_time - start_time
    bubble_sort_times.append(time_taken)
    print(f"Time taken for bubble sort on array {index}: {time_taken:.6f} seconds")

# Calculate and print average bubble sort time
avg_bubble_sort_time = sum(bubble_sort_times) / len(bubble_sort_times)
print(f"\nAverage time taken for bubble sort: {avg_bubble_sort_time:.6f} seconds")

```

Array size = 100000

## Implementation of Optimized Bubble Sort for larger arrays

```
#Optimized bubble sort implementation
#Array size = 1000
import time
import copy # To make copies of the arrays

# Optimized Bubble Sort function
def optimized_bubble(arr):
    n = len(arr)
    for i in range(n):
        swapped = False
        for j in range(n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j] # Swap elements
                swapped = True
        if not swapped:
            break

# ----- OPTIMIZED BUBBLE SORT TIMING -----
optimized_bubble_sort_times = []

for index, original_array in enumerate(original_bubble_arrays_1000, 1):
    arr_copy = copy.deepcopy(original_array) # Copy to preserve the original array

    start_time = time.time()
    optimized_bubble(arr_copy)
    end_time = time.time()

    time_taken = end_time - start_time
    optimized_bubble_sort_times.append(time_taken)
    print(f"Time taken for optimized bubble sort on array {index}: {time_taken:.6f} seconds")

# Calculate and print average optimized bubble sort time
avg_optimized_bubble_sort_time = sum(optimized_bubble_sort_times) / len(optimized_bubble_sort_times)
print(f"\nAverage time taken for optimized bubble sort: {avg_optimized_bubble_sort_time:.6f} seconds")
```

Array size = 1000

```
#Array size = 100000
import time
import copy # To make copies of the arrays

# Optimized Bubble Sort function
def optimized_bubble(arr):
    n = len(arr)
    for i in range(n):
        swapped = False
        for j in range(n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j] # Swap elements
                swapped = True
        if not swapped:
            break

# ----- OPTIMIZED BUBBLE SORT TIMING -----
optimized_bubble_sort_times = []

for index, original_array in enumerate(original_bubble_arrays_100000, 1):
    arr_copy = copy.deepcopy(original_array) # Copy to preserve the original array

    start_time = time.time()
    optimized_bubble(arr_copy)
    end_time = time.time()

    time_taken = end_time - start_time
    optimized_bubble_sort_times.append(time_taken)
    print(f"Time taken for optimized bubble sort on array {index}: {time_taken:.6f} seconds")

# Calculate and print average optimized bubble sort time
avg_optimized_bubble_sort_time = sum(optimized_bubble_sort_times) / len(optimized_bubble_sort_times)
print(f"\nAverage time taken for optimized bubble sort: {avg_optimized_bubble_sort_time:.6f} seconds")
```

Array size = 100000

## Implementation of Insertion Sort for larger arrays

```
#Insertion sort implementation
#Array size = 1000
import time
import copy # To make copies of the arrays

# Insertion Sort function
def insertion_sort(list_a):
    for i in range(1, len(list_a)): # Start from the second element
        value_to_sort = list_a[i]
        j = i - 1
        while j >= 0 and list_a[j] > value_to_sort:
            list_a[j + 1] = list_a[j] # Shift elements to the right
            j -= 1
        list_a[j + 1] = value_to_sort # Insert value in the correct position

# ----- INSERTION SORT TIMING -----
insertion_sort_times = []

for index, original_array in enumerate(original_bubble_arrays_1000, 1):
    arr_copy = copy.deepcopy(original_array) # Copy to preserve the original array

    start_time = time.time()
    insertion_sort(arr_copy)
    end_time = time.time()

    time_taken = end_time - start_time
    insertion_sort_times.append(time_taken)
    print(f"Time taken for insertion sort on array {index}: {time_taken:.6f} seconds")

# Calculate and print average insertion sort time
avg_insertion_sort_time = sum(insertion_sort_times) / len(insertion_sort_times)
print(f"\nAverage time taken for insertion sort: {avg_insertion_sort_time:.6f} seconds")
```

Array size = 1000

```
#Array size = 100000
import time
import copy # To make copies of the arrays

# Insertion Sort function
def insertion_sort(list_a):
    for i in range(1, len(list_a)): # Start from the second element
        value_to_sort = list_a[i]
        j = i - 1
        while j >= 0 and list_a[j] > value_to_sort:
            list_a[j + 1] = list_a[j] # Shift elements to the right
            j -= 1
        list_a[j + 1] = value_to_sort # Insert value in the correct position

# ----- INSERTION SORT TIMING -----
insertion_sort_times = []

for index, original_array in enumerate(original_bubble_arrays_100000, 1):
    arr_copy = copy.deepcopy(original_array) # Copy to preserve the original array

    start_time = time.time()
    insertion_sort(arr_copy)
    end_time = time.time()

    time_taken = end_time - start_time
    insertion_sort_times.append(time_taken)
    print(f"Time taken for insertion sort on array {index}: {time_taken:.6f} seconds")

# Calculate and print average insertion sort time
avg_insertion_sort_time = sum(insertion_sort_times) / len(insertion_sort_times)
print(f"\nAverage time taken for insertion sort: {avg_insertion_sort_time:.6f} seconds")
```

Array size = 100000

## Implementation of Selection Sort for larger arrays

```
#Selection sort implementation
# Array size = 1000
import time
import copy # To make copies of the arrays

# Selection Sort function
def selection_sort(list_a):
    for i in range(len(list_a) - 1):
        min_index = i # Assume the current element is the smallest

        for j in range(i + 1, len(list_a)):
            if list_a[j] < list_a[min_index]:
                min_index = j # Update the index of the smallest element

        if min_index != i:
            list_a[i], list_a[min_index] = list_a[min_index], list_a[i] # Swap values

# ----- SELECTION SORT TIMING -----
selection_sort_times = []

for index, original_array in enumerate(original_bubble_arrays_1000, 1):
    arr_copy = copy.deepcopy(original_array) # Copy to preserve the original array

    start_time = time.time()
    selection_sort(arr_copy)
    end_time = time.time()

    time_taken = end_time - start_time
    selection_sort_times.append(time_taken)
    print(f"Time taken for selection sort on array {index}: {time_taken:.6f} seconds")

# Calculate and print average selection sort time
avg_selection_sort_time = sum(selection_sort_times) / len(selection_sort_times)
print(f"\nAverage time taken for selection sort: {avg_selection_sort_time:.6f} seconds")
```

Array size = 1000

```
# Array size = 100000
import time
import copy # To make copies of the arrays

# Selection Sort function
def selection_sort(list_a):
    for i in range(len(list_a) - 1):
        min_index = i # Assume the current element is the smallest

        for j in range(i + 1, len(list_a)):
            if list_a[j] < list_a[min_index]:
                min_index = j # Update the index of the smallest element

        if min_index != i:
            list_a[i], list_a[min_index] = list_a[min_index], list_a[i] # Swap values

# ----- SELECTION SORT TIMING -----
selection_sort_times = []

for index, original_array in enumerate(original_bubble_arrays_100000, 1):
    arr_copy = copy.deepcopy(original_array) # Copy to preserve the original array

    start_time = time.time()
    selection_sort(arr_copy)
    end_time = time.time()

    time_taken = end_time - start_time
    selection_sort_times.append(time_taken)
    print(f"Time taken for selection sort on array {index}: {time_taken:.6f} seconds")

# Calculate and print average selection sort time
avg_selection_sort_time = sum(selection_sort_times) / len(selection_sort_times)
print(f"\nAverage time taken for selection sort: {avg_selection_sort_time:.6f} seconds")
```

Array size = 100000

## Comparison of non-comparison-based algorithms

### Implementation of Counting Sort for larger arrays

```
#Counting sort implementation
#Array size = 1000
import random
import time
import copy # To make copies of the arrays

# Counting Sort function
def counting_sort(arr):
    max_value = max(arr)
    min_value = min(arr)

    range_of_values = max_value - min_value + 1
    count = [0] * range_of_values
    output = [0] * len(arr)

    for num in arr:
        count[num - min_value] += 1

    for i in range(1, range_of_values):
        count[i] += count[i - 1]

    for num in reversed(arr):
        output[count[num - min_value] - 1] = num
        count[num - min_value] -= 1

    # Copy sorted values back into original array
    for i in range(len(arr)):
        arr[i] = output[i]
```

```
# ----- GENERATING NEW ARRAYS -----
num_arrays = 3
array_size = 1000
original_counting_sort_arrays_1000 = [[random.randint(1, 1000) for _ in range(array_size)] for _ in range(num_arrays)]

# ----- COUNTING SORT TIMING -----
counting_sort_times = []

for index, original_array in enumerate(original_counting_sort_arrays_1000, 1):
    arr_copy = copy.deepcopy(original_array) # Copy to preserve the original array

    start_time = time.time()
    counting_sort(arr_copy)
    end_time = time.time()

    time_taken = end_time - start_time
    counting_sort_times.append(time_taken)
    print(f"Time taken for counting sort on array {index}: {time_taken:.6f} seconds")

# Calculate and print average counting sort time
avg_counting_sort_time = sum(counting_sort_times) / len(counting_sort_times)
print(f"\nAverage time taken for counting sort: {avg_counting_sort_time:.6f} seconds")
```

Array size = 1000

```

#Array size = 100000
import random
import time
import copy # To make copies of the arrays

# Counting Sort function
def counting_sort(arr):
    max_value = max(arr)
    min_value = min(arr)

    range_of_values = max_value - min_value + 1
    count = [0] * range_of_values
    output = [0] * len(arr)

    for num in arr:
        count[num - min_value] += 1

    for i in range(1, range_of_values):
        count[i] += count[i - 1]

    for num in reversed(arr):
        output[count[num - min_value] - 1] = num
        count[num - min_value] -= 1

    for i in range(len(arr)):
        arr[i] = output[i]

# ----- GENERATING NEW ARRAYS -----
num_arrays = 3
array_size = 100000
original_counting_sort_arrays_100000 = [[random.randint(1, 100000) for _ in range(array_size)] for _ in range(num_arrays)]

# ----- COUNTING SORT TIMING -----
counting_sort_times = []

for index, original_array in enumerate(original_counting_sort_arrays_100000, 1):
    arr_copy = copy.deepcopy(original_array) # Copy to preserve the original array

    start_time = time.time()
    counting_sort(arr_copy)
    end_time = time.time()

    time_taken = end_time - start_time
    counting_sort_times.append(time_taken)
    print(f"Time taken for counting sort on array {index}: {time_taken:.6f} seconds")

# Calculate and print average counting sort time
avg_counting_sort_time = sum(counting_sort_times) / len(counting_sort_times)
print(f"\nAverage time taken for counting sort: {avg_counting_sort_time:.6f} seconds")

```

Array size = 100000

## Implementation of Bucket Sort for larger arrays

```
#Bucket sort implementation
# Array size = 1000
import random
import time
import copy # To make copies of the arrays

# Bucket Sort function
def bucket_sort(array):
    bucket_count = len(array) # Number of buckets
    bucket = [[] for _ in range(bucket_count)] # Create empty buckets

    # Insert elements into buckets
    for num in array:
        index_b = min(int(bucket_count * (num / 1000)), bucket_count - 1) # Fix IndexError
        bucket[index_b].append(num)

    # Sort each bucket and concatenate results
    sorted_array = []
    for i in range(bucket_count):
        bucket[i] = sorted(bucket[i]) # Use Python's built-in sort
        sorted_array.extend(bucket[i])

    return sorted_array

# ----- REUSING COUNTING SORT ARRAYS -----
original_bucket_sort_arrays_1000 = copy.deepcopy(original_counting_sort_arrays_1000)

# ----- BUCKET SORT TIMING -----
bucket_sort_times = []

for index, original_array in enumerate(original_bucket_sort_arrays_1000, 1):
    arr_copy = copy.deepcopy(original_array) # Copy to preserve the original array

    start_time = time.time()
    sorted_arr = bucket_sort(arr_copy)
    end_time = time.time()

    time_taken = end_time - start_time
    bucket_sort_times.append(time_taken)
    print(f"Time taken for bucket sort on array {index}: {time_taken:.6f} seconds")

# Calculate and print average bucket sort time
avg_bucket_sort_time = sum(bucket_sort_times) / len(bucket_sort_times)
print(f"\nAverage time taken for bucket sort: {avg_bucket_sort_time:.6f} seconds")
```

Array size = 1000

```

# Array size = 100000
import random
import time
import copy # To make copies of the arrays

# Bucket Sort function
def bucket_sort(array):
    bucket_count = len(array) # Number of buckets
    bucket = [[] for _ in range(bucket_count)] # Create empty buckets

    # Insert elements into buckets
    for num in array:
        index_b = min(int(bucket_count * (num / 100000)), bucket_count - 1) # Fix IndexError
        bucket[index_b].append(num)

    # Sort each bucket and concatenate results
    sorted_array = []
    for i in range(bucket_count):
        bucket[i] = sorted(bucket[i]) # Use Python's built-in sort
        sorted_array.extend(bucket[i])

    return sorted_array

# ----- REUSING COUNTING SORT ARRAYS -----
original_bucket_sort_arrays_100000 = copy.deepcopy(original_counting_sort_arrays_100000)

# ----- BUCKET SORT TIMING -----
bucket_sort_times = []

for index, original_array in enumerate(original_bucket_sort_arrays_100000, 1):
    arr_copy = copy.deepcopy(original_array) # Copy to preserve the original array

    start_time = time.time()
    sorted_arr = bucket_sort(arr_copy)
    end_time = time.time()

    time_taken = end_time - start_time
    bucket_sort_times.append(time_taken)
    print(f"Time taken for bucket sort on array {index}: {time_taken:.6f} seconds")

# Calculate and print average bucket sort time
avg_bucket_sort_time = sum(bucket_sort_times) / len(bucket_sort_times)
print(f"\nAverage time taken for bucket sort: {avg_bucket_sort_time:.6f} seconds")

```

Array size = 100000

## Implementation of Radix Sort for larger arrays

```
#Radix sort implementation
#Array size = 1000
import time
import copy # To make copies of the arrays

# Counting Sort used in Radix Sort
def counting_sort(l, place):
    size = len(l)
    output = [0] * size
    count = [0] * 10

    for i in range(size):
        index = (l[i] // place) % 10
        count[index] += 1

    for j in range(1, 10):
        count[j] += count[j-1]

    k = size - 1
    while k >= 0:
        index = (l[k] // place) % 10
        output[count[index] - 1] = l[k]
        count[index] -= 1
        k -= 1
```

```
    for a in range(size):
        l[a] = output[a]

# Radix Sort function
def radix_sort(l):
    max_val = max(l)
    place = 1
    while max_val // place > 0:
        counting_sort(l, place)
        place *= 10
```

```
# ----- REUSING COUNTING SORT ARRAYS -----
original_radix_sort_arrays_1000 = copy.deepcopy(original_counting_sort_arrays_1000)

# ----- RADIX SORT TIMING -----
radix_sort_times = []

for index, original_array in enumerate(original_radix_sort_arrays_1000, 1):
    arr_copy = copy.deepcopy(original_array) # Copy to preserve the original array

    start_time = time.time()
    radix_sort(arr_copy)
    end_time = time.time()

    time_taken = end_time - start_time
    radix_sort_times.append(time_taken)
    print(f"Time taken for radix sort on array {index}: {time_taken:.6f} seconds")

# Calculate and print average radix sort time
avg_radix_sort_time = sum(radix_sort_times) / len(radix_sort_times)
print(f"\nAverage time taken for radix sort (Array Size = 1000): {avg_radix_sort_time:.6f} seconds")
```

Array size = 1000

```

#Array size = 100000
import time
import copy # To make copies of the arrays

# Counting Sort used in Radix Sort
def counting_sort(l, place):
    size = len(l)
    output = [0] * size
    count = [0] * 10

    for i in range(size):
        index = (l[i] // place) % 10
        count[index] += 1

    for j in range(1, 10):
        count[j] += count[j-1]

    k = size - 1
    while k >= 0:
        index = (l[k] // place) % 10
        output[count[index] - 1] = l[k]
        count[index] -= 1
        k -= 1

```

```

        for a in range(size):
            l[a] = output[a]

# Radix Sort function
def radix_sort(l):
    max_val = max(l)
    place = 1
    while max_val // place > 0:
        counting_sort(l, place)
        place *= 10

```

```

# ----- REUSING COUNTING SORT ARRAYS -----
original_radix_sort_arrays_100000 = copy.deepcopy(original_counting_sort_arrays_100000)

# ----- RADIX SORT TIMING -----
radix_sort_times = []

for index, original_array in enumerate(original_radix_sort_arrays_100000, 1):
    arr_copy = copy.deepcopy(original_array) # Copy to preserve the original array

    start_time = time.time()
    radix_sort(arr_copy)
    end_time = time.time()

    time_taken = end_time - start_time
    radix_sort_times.append(time_taken)
    print(f"Time taken for radix sort on array {index}: {time_taken:.6f} seconds")

# Calculate and print average radix sort time
avg_radix_sort_time = sum(radix_sort_times) / len(radix_sort_times)
print(f"\nAverage time taken for radix sort (Array Size = 100000): {avg_radix_sort_time:.6f} seconds")

```

Array size = 100000

## REFERENCES

- [1] K. Goel, P. Dwivedi, and O. Sharma, "Performance Analysis of Various Sorting Algorithms: Comparison and Optimization," Dec. 2023,  
doi: <https://doi.org/10.1109/ised59382.2023.10444609>.
- [2] O. K. Durrani and S. AbdulHayan, "Performance Measurement of Popular Sorting Algorithms Implemented using Java and Python," *2022 International Conference on Electrical, Computer, Communications and Mechatronics Engineering (ICECCME)*, Nov. 2022,  
doi: <https://doi.org/10.1109/iceccme55909.2022.9988424>.
- [3] J. Lobo and S. Kuwelkar, "Performance Analysis of Merge Sort Algorithms," *IEEE Xplore*, Jul. 01, 2020. <https://ieeexplore.ieee.org/abstract/document/9155623>
- [4] S. S. Skiena, *Algorithm Design Manual*. S.L.: Springer Nature, 2021.
- [5] N. Karumanchi, *Data Structures and Algorithms*.
- [6] A. Bhargava, *Grokking Algorithms*. Simon and Schuster, 2016.
- [7] T. Niemann, *Sorting and Searching Algorithms*.
- [8] J. Hammad, "A Comparative Study between Various Sorting Algorithms," *International Journal of Computer Science and Network Security (IJCSNS)*, vol. 15, no. 3, pp. XX-XX, 2015.
- [9] R. Sedgewick, *Algorithms in C*. Addison-Wesley Professional, 1998.
- [10] K. Mehlhorn, *Data Structures and Algorithms 1*. Springer Science & Business Media, 2013.
- [11] D. R. Babu, R. S. Shankar, V. P. Kumar, C. S. Rao, D. M. Babu and V. C. Sekhar, "Array-indexed sorting algorithm for natural numbers," *2011 IEEE 3rd International Conference on Communication Software and Networks*, Xi'an, China, 2011, pp. 606-609,  
doi: 10.1109/ICCSN.2011.6014966.
- [12] R. Sedgewick and K. Wayne, *Algorithms*. Addison-Wesley Professional, 2014.
- [13] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to algorithms*. MIT Press, 2009.
- [14] Niklaus Wirth, *Algorithms + data structures = programs*. New Delhi: Prentice-Hall Of India, 2008.